Computer Networks

Chapter 6 Transport Layer

大纲

- 传输服务
- 传输协议的要素
- 拥塞控制
- 因特网传输协议UDP
- 因特网传输协议TCP
- 性能问题
- 延迟容忍网络

6.1 概述

- 引入传输层的原因
  - 消除网络层的不可靠性；
  - 提供从源端主机到目的端主机的可靠的、与实际使用的网络无关的信息传输。
- 传输服务
  - 传输实体（transport entity）：完成传输层功能的硬软件；
  - 传输层实体利用网络层提供的服务向高层提供有效、可靠的服务；
  - 传输层提供两种服务
    - 面向连接的传输服务：连接建立，数据传输，连接释放
    - 无连接的传输服务。
  - 1～4层称为传输服务提供者（transport service provider），4层以上称为传输服务用户（transport service user）。

6.1 传输服务

- 提供服务
  - 传输实体（transport entity）：完成传输层功能的硬软件；
  - 传输层实体利用网络层提供的服务向高层提供有效、可靠的服务；
  - 传输层提供两种服务
    - 面向连接的传输服务：连接建立，数据传输，连接释放
    - 无连接的传输服务。
    - 图6-1(P382)

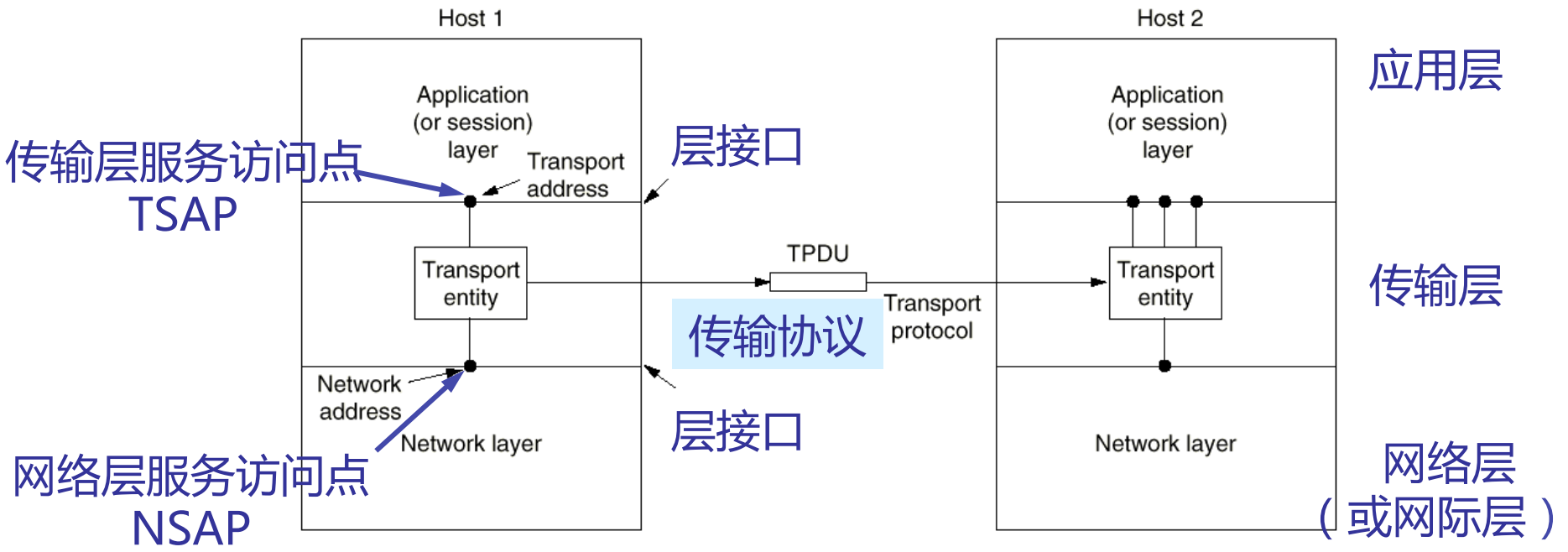
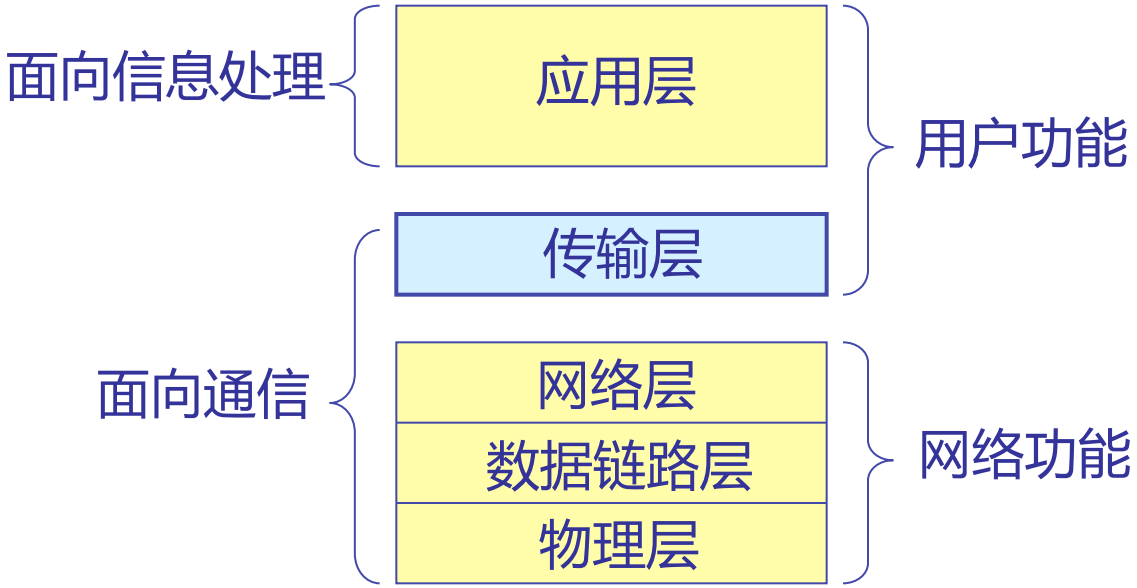


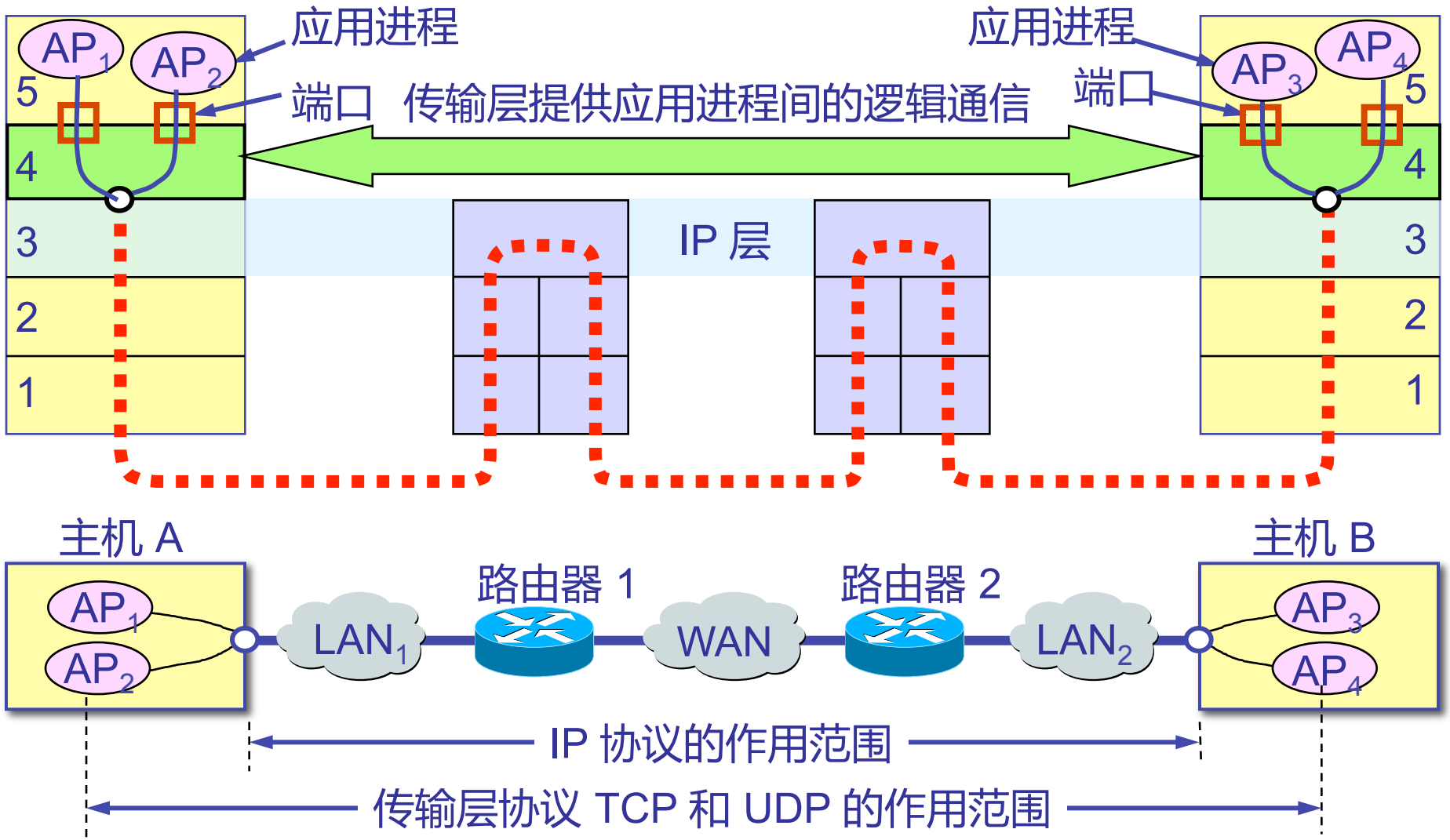
Fig. 6-1. The network, transport, and application layers.

6.1 传输层协议概述

- 从通信和信息处理的角度看，传输层向它上面的应用层提供通信服务，它属于面向通信部分的最高层，同时也是用户功能中的最低层。



传输层为相互通信的应用进程提供了逻辑通信



应用进程之间的通信

- 两个主机进行通信实际上就是两个主机中的应用进程互相通信。
- 应用进程之间的通信又称为端到端的通信。
- 传输层的一个很重要的功能就是复用和分用。应用层不同进程的报文通过不同的端口向下交到传输层，再往下就共用网络层提供的服务。
- “传输层提供应用进程间的逻辑通信”。
- “逻辑通信”的意思是：传输层之间的通信好像是沿水平方向传送数据。但事实上这两个传输层之间并没有一条水平方向的物理连接。

提供给上层的服务

- 提供服务
  - 传输实体：
  - 面向连接服务/无连接服务
    - 传输层将子网的技术、设计和缺陷与上层隔离
  - 1~4传输服务提供者，5~7传输服务用户
    - 传输层将增强网络层的服务质量

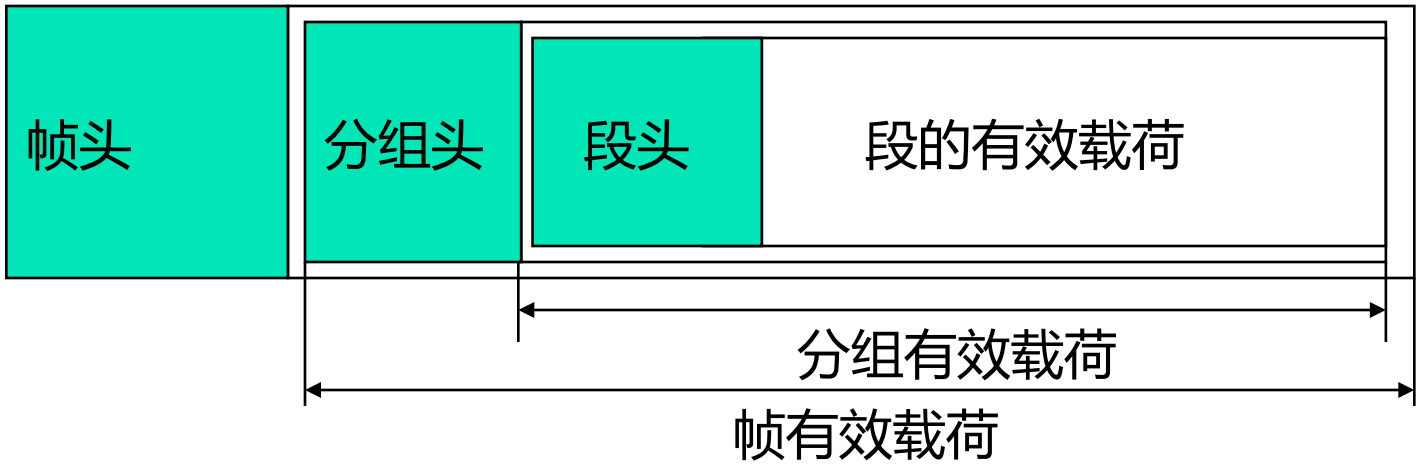
传输服务原语

- 传输层与网络层
  - 传输服务屏蔽不同的网络
    - 面向连接服务：不可靠的网络服务上建立可靠服务
    - 无连接服务
  - 服务对象：程序只能看到传输服务，无法控制网络
- 传输服务原语

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

传输服务原语(2)

- 原语
  - LISTEN
  - CONNECT
  - SEND
  - RECEIVE
  - DISCONNECT
- 段（传输协议数据单元TPDU）—传输实体间发送的消息



- 释放连接：非对称方式，对称方式

Berkeley 套接字—另一组传输原语

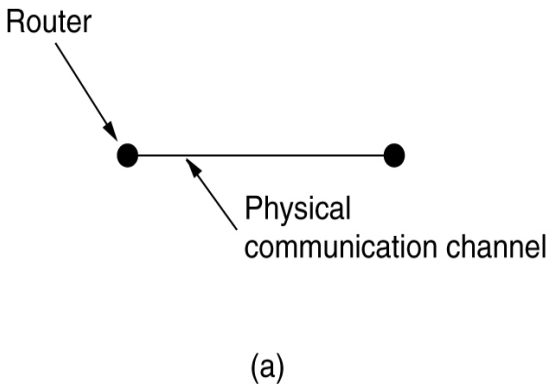
- TCP所用套接字(socket)原语
  - UNIX，Windows(winsock)

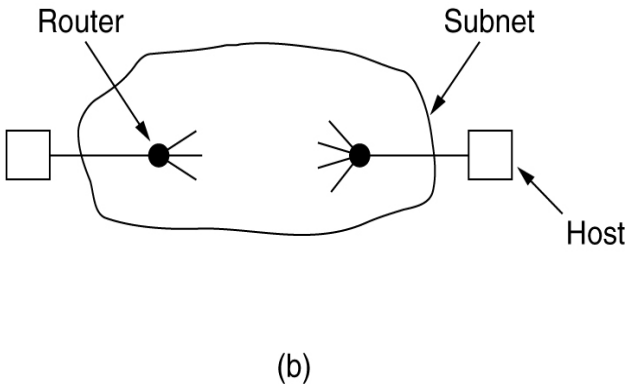
原语	含义
SOCKET	创建一个新通信端点
BIND	将套接字与一个本地地址关联
LISTEN	声明愿意接受连接，给出队列长度
ACCEPT	阻塞调用者，直到入境连接到来
CONNECT	创建一个链接
SEND	通过连接发送一些数据
RECEIVE	从连接接收一些数据
CLOSE	释放连接

套接字编程实例：Internet文件服务器

6.2 传输协议的要素

- 与数据链路层类似
  - 相同点：处理错误控制、顺序性、流控制
  - 不同点：指定目的地址、初始连接更为复杂、子网存储能力导致乱序接收、缓冲区数量可变
- 要素
  1. 寻址
  2. 建立连接
  3. 释放连接
  4. 流量控制和缓冲策略
  5. 多路复用
  6. 崩溃恢复





6.2.1 寻址

- 方法：定义传输服务访问点TSAP（Transport Service Access Point），将应用进程与这些TSAP相连。在Internet中，TSAP为（IP address, local port）；
- Fig6-8
- 远方客户程序如何获得服务程序的TSAP？
  - 方法1：预先约定、广为人知的，像telnet是（IP地址，端口23）；
  - 方法2：从名字服务器（name server）或目录服务器（directory server）获得TSAP
    - 一个特殊的进程称为**名字服务器**或**目录服务器**（TSAP众所周知）；
    - 用户与名字服务器建立连接，发送服务名称，获得服务进程的TSAP，释放与名称服务器的连接；
    - 与服务进程建立连接。

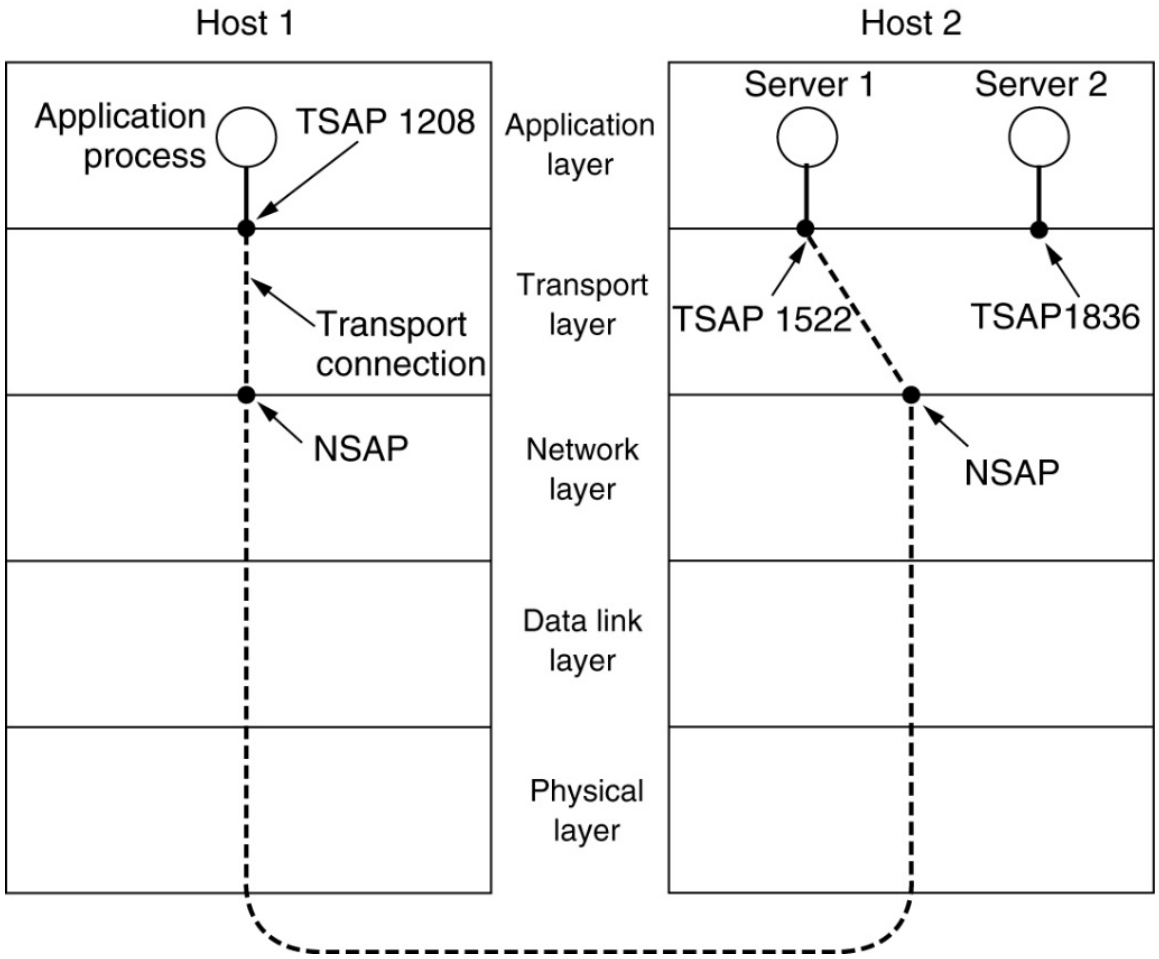


Fig6-8　TSAP、NSAP和传输连接

初始连接协议

- 当服务程序很多时，使用初始连接协议（initial connection protocol）
  - 一个称为**进程服务器（process server）**的进程(inetd)同时在多个端口上监听；
  - 远方客户程序向它实际想访问的服务程序的TSAP发出连接建立请求；
  - 如果没有服务程序在此TSAP上监听，则远方客户和进程服务器建立连接；
  - 进程服务器产生所请求的服务进程，并使该进程继承和远程客户的连接；
  - 进程服务器返回继续监听；
  - 远方客户程序与所希望的服务程序进行数据传输。
  - Fig. 6-9

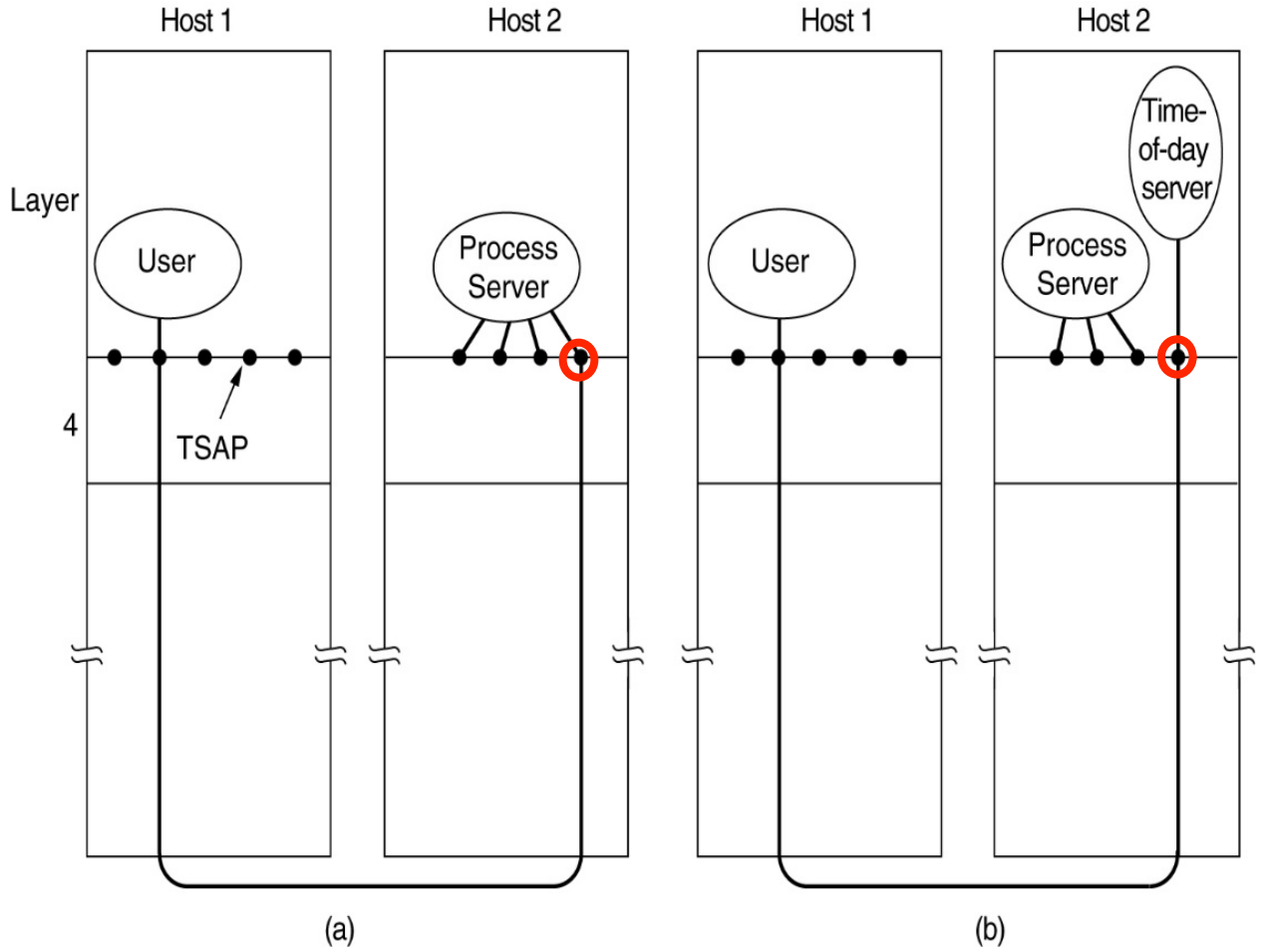


Fig6-9 主机1上的进程通过进程服务器与主机2上的被请求服务器建立连接

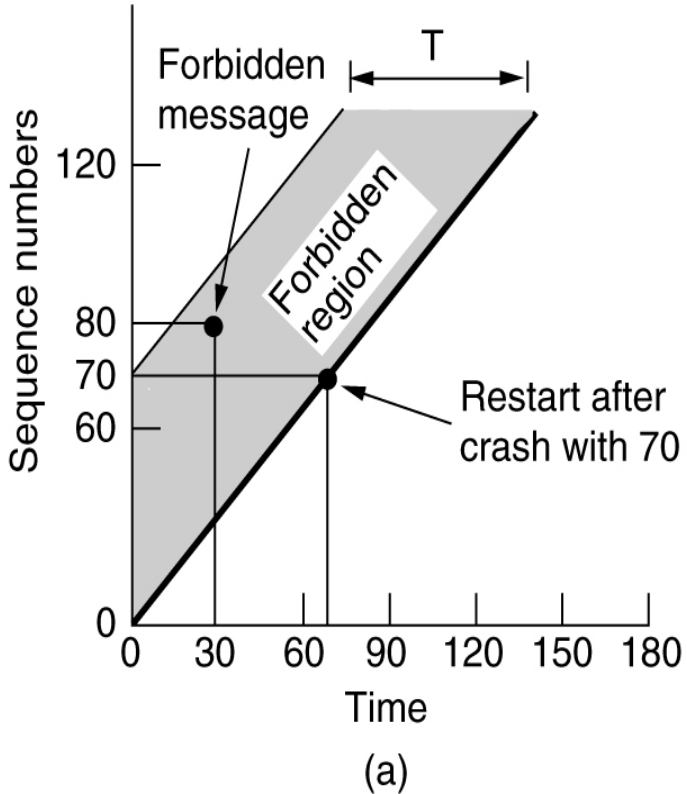
17

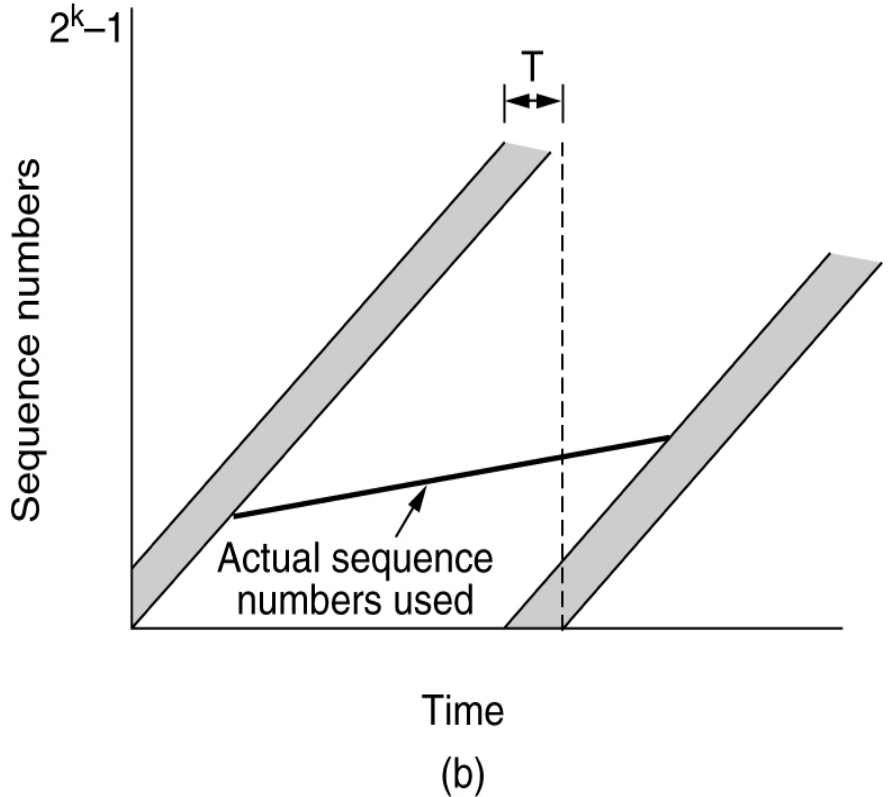
6.2.2 建立连接

- 网络问题：分组可能丢失、延迟、损坏、重复出现
  - 废弃使用过的传输地址
  - 分配一个连接标识符
- 限制数据包在网络中的生存期
  - 限制网络设计
  - 放置跳计数器
  - 时间戳
- 区分重复段
  - 段标签：序号在周期T秒内不重复
  - 日时钟：永远运行，时钟低位作为开始序号

日时钟方案

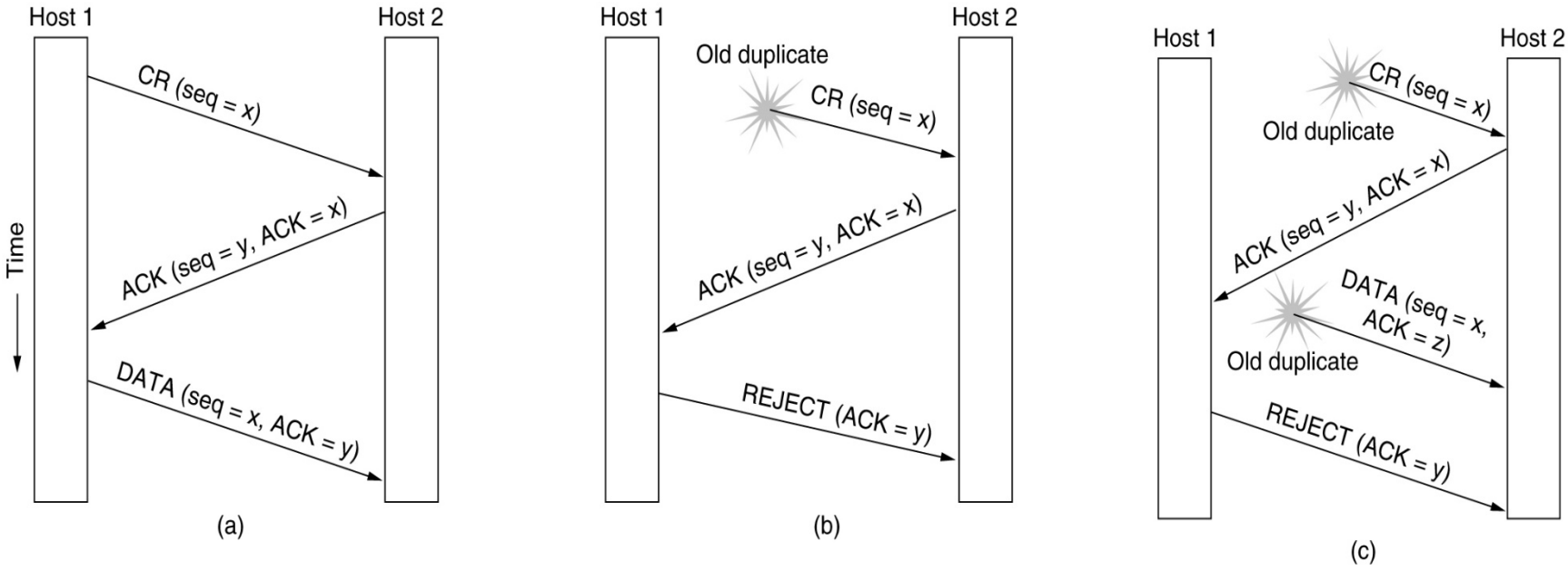
- 禁止区域是主机崩溃后，段序号的取值区间，但这段区间的序号不可能使用
- 防止序号进入禁止区域
  - 发送速度不能太快
  - 发送速率不能低于时钟速率





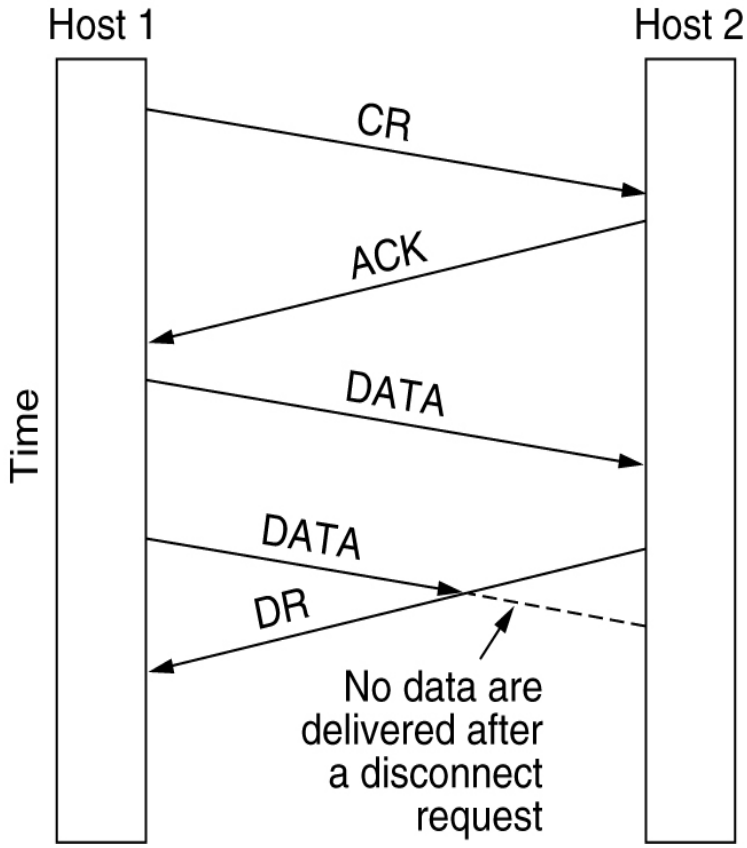
建立连接(2)

- 三次握手
  - 时间戳：辅助扩展32位序号
  - 防止序号回绕(PASW)：伪随机初始序号
    - 防止攻击者预测到下一个初始序号



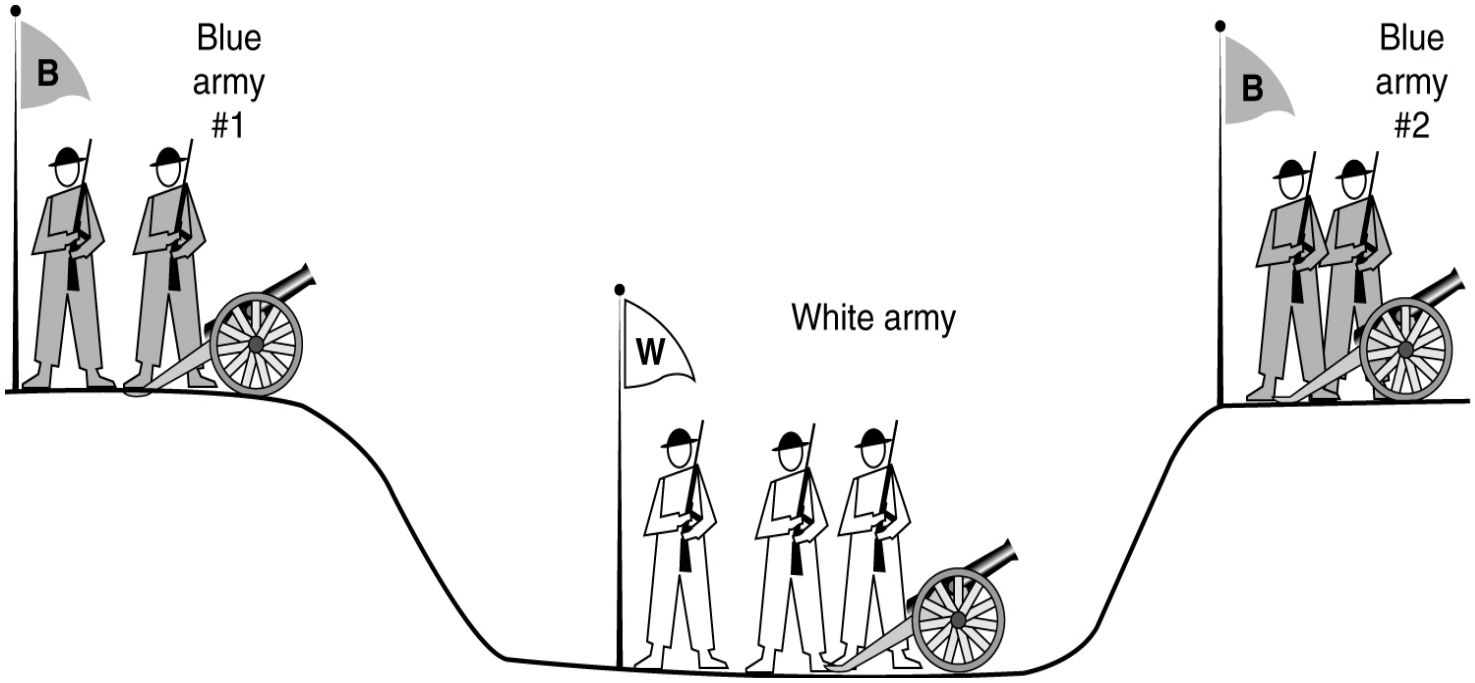
6.2.3 释放连接

- 释放
  - 非对称
    - 任何一方可DISCONNECT，其TPDU到达对方-连接释放
    - 右图导致数据丢失
  - 对称：
    - 各方单独DISCONNECT，停止发送但继续接收
    - 双方均DISCONNECT后释放连接



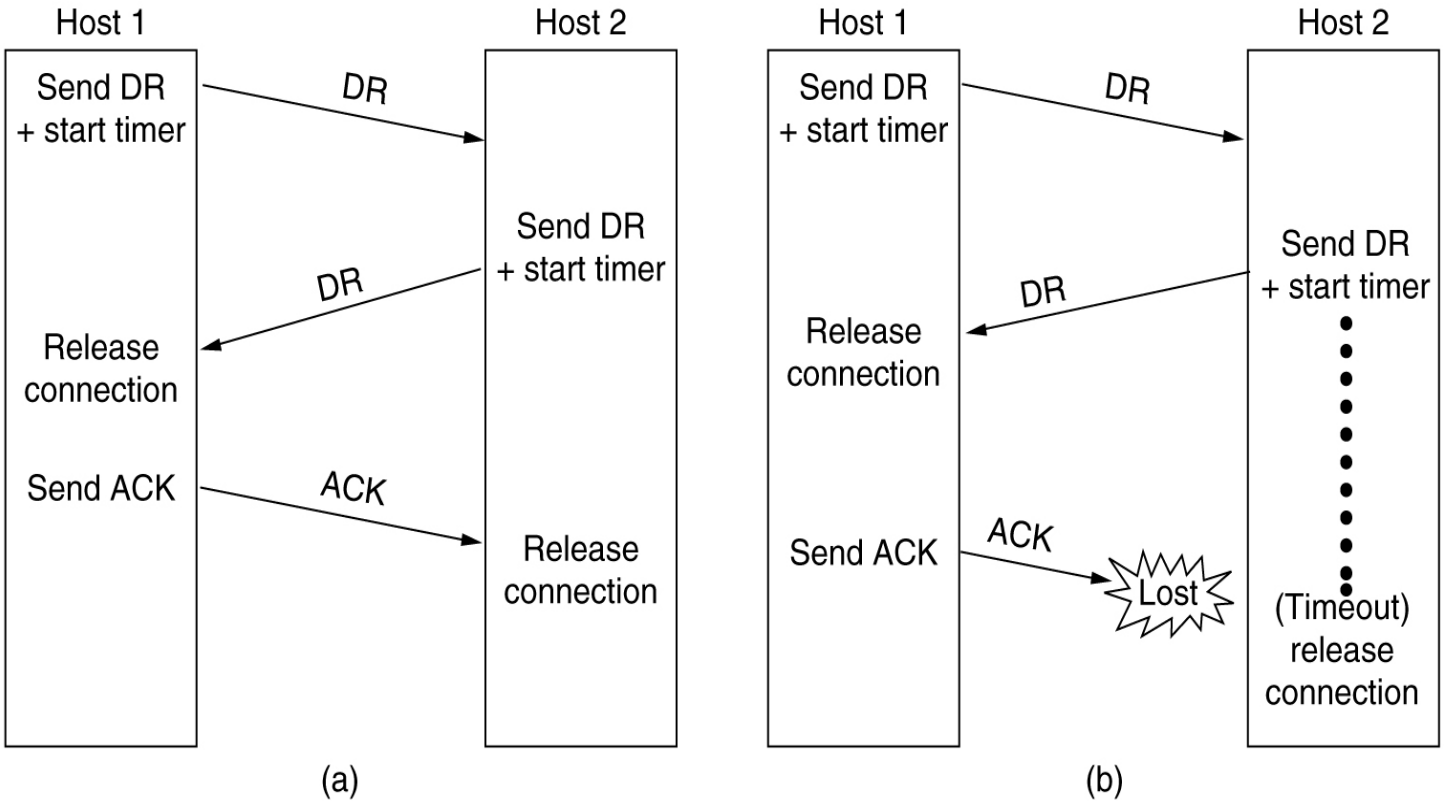
释放连接(2)

- 两军问题



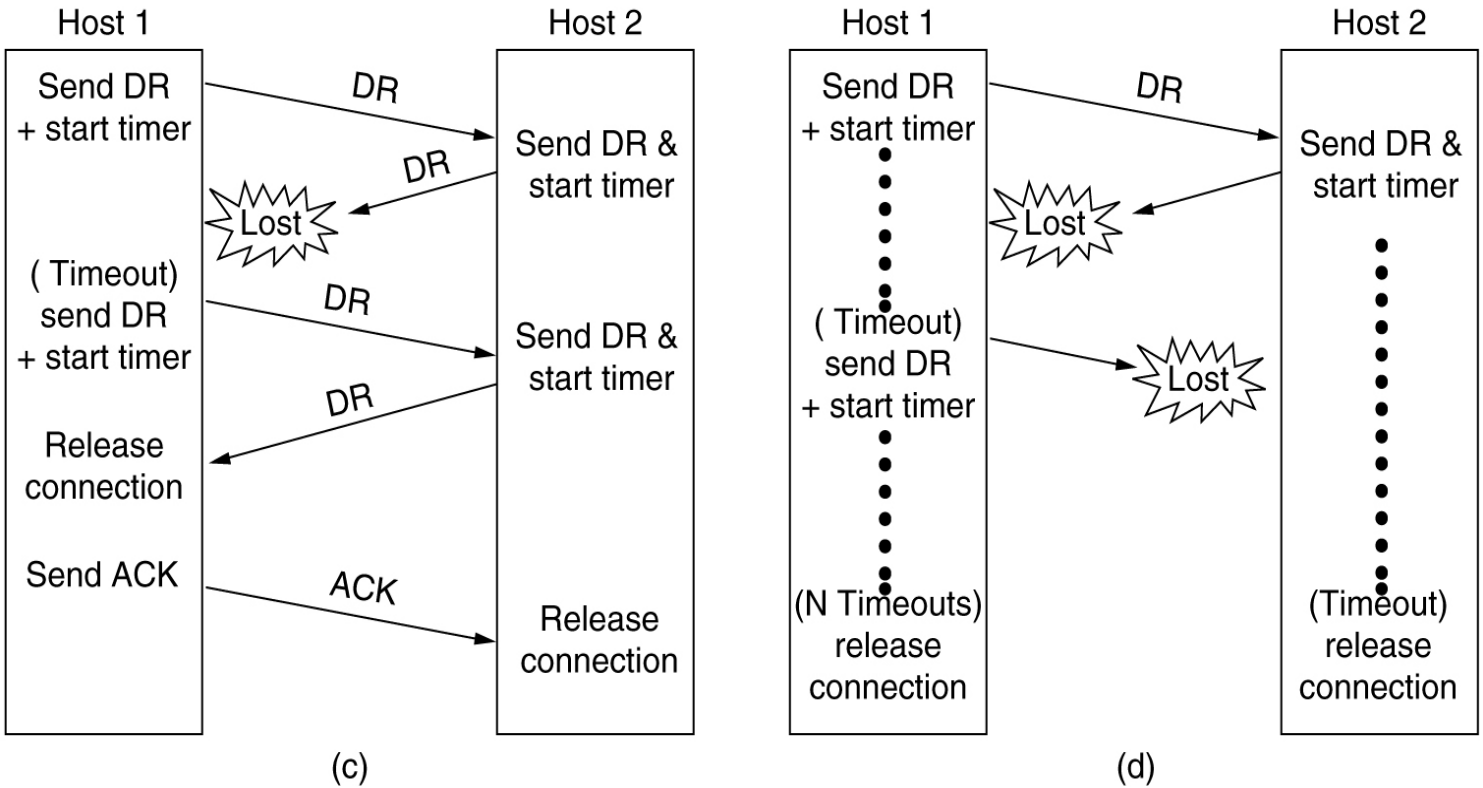
释放连接(3)

- 释放连接的协议场景
  - 正常情况
  - 超时释放



释放连接(4)

- 释放连接的协议场景
  - 第二次成功
  - 重传全部失败



进一步的讨论

- 如果初始DR和N次重传全部丢失，则协议失败
  - 发送端放弃发送强行释放，导致半开放连接
- 不允许发送端放弃
  - 允许超时—导致永久释放操作
  - 不允许超时—协议将被悬挂

- 传输实体本身无法彻底解决这一问题

6.2.4 差错控制和流量控制

- 差错控制
  - 确保数据传输的可靠性
- 流量控制
  - 防止快速发送端淹没慢速接收端
- 与数据链路层协议不同
  - 数据链路层
    - CRC、自动重发请求、停等、滑动窗口
  - 功能上的差异
    - 链路层校验和只保护在一条链路上进过的数据包，路由器错误在链路层校验之外；相比链路层校验，端到端的传输层校验至关重要
  - 程度上的差异
    - 链路的时延带宽积很小，有线与光纤链路误码很低可忽略重传；
    - 传输层连接的时延带宽积较大，需使用较大的滑动窗口

差错控制和流量控制

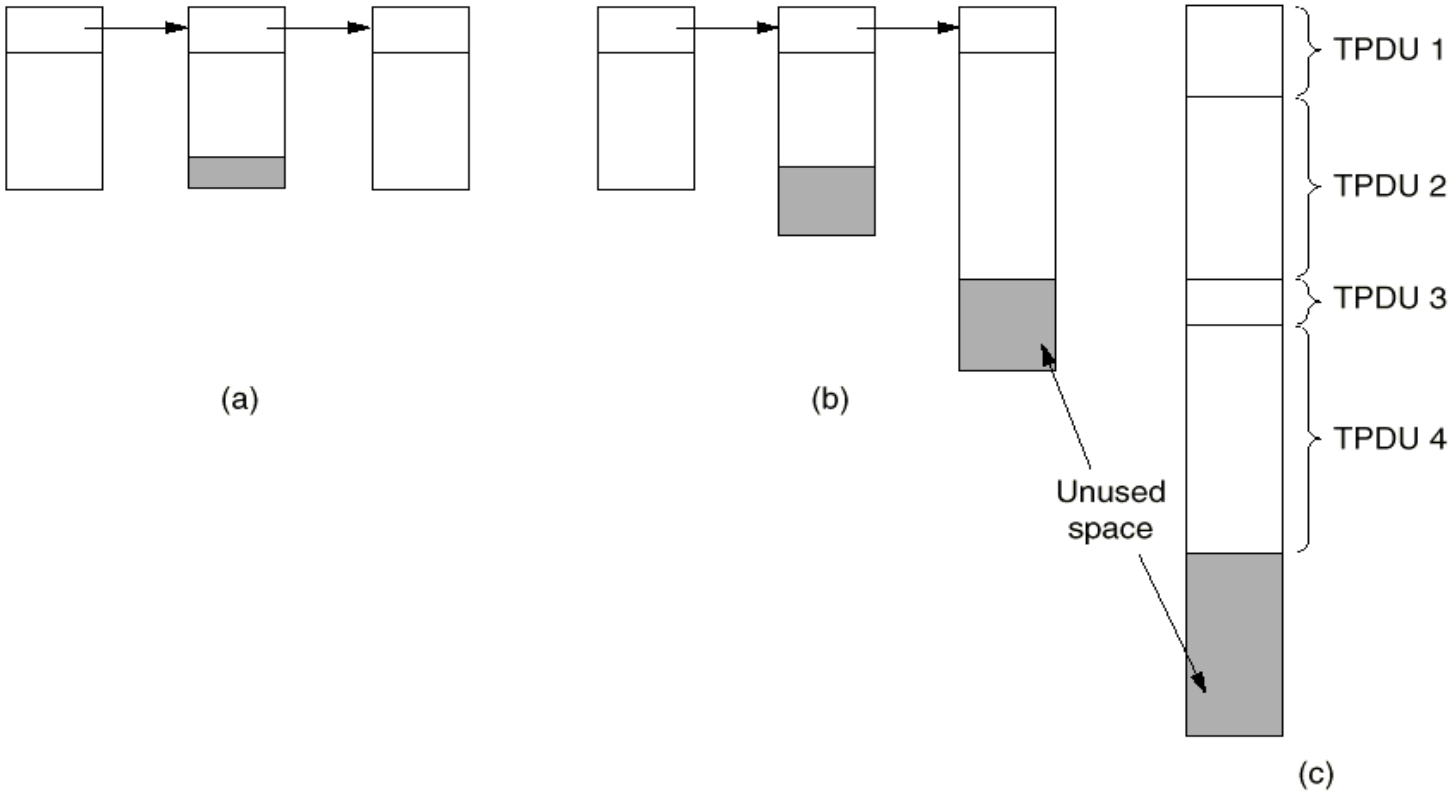
- 流控和缓存（Flow Control and Buffering）
  - 缓存：由于网络层服务是不可靠的，传输层实体必须缓存所有连接发出的TPDU，而且为每个连接单独做缓存，以便用于错误情况下的重传。接收方的传输层实体既可以做也可以不做缓存。缓存区的设计有三种。

    Fig. 6-15
  - 流控：传输层利用可变滑动窗口协议来实现流控。所谓可变滑动窗口协议，是指发送方的发送窗口大小是由接收方根据自己的实际缓存情况给出的。为了避免控制TPDU丢失导致死锁，主机应该周期性的发送TPDU。

    Fig. 6-16

流量控制和缓冲策略

- 缓冲池的组织方式
  - a. 段的长度都差不多——链接固定大小的缓冲区
  - b. 段的长度差异较大—— 链接可变大小的缓冲区—管理复杂
  - c. 每条连接使用一个大的循环缓冲区



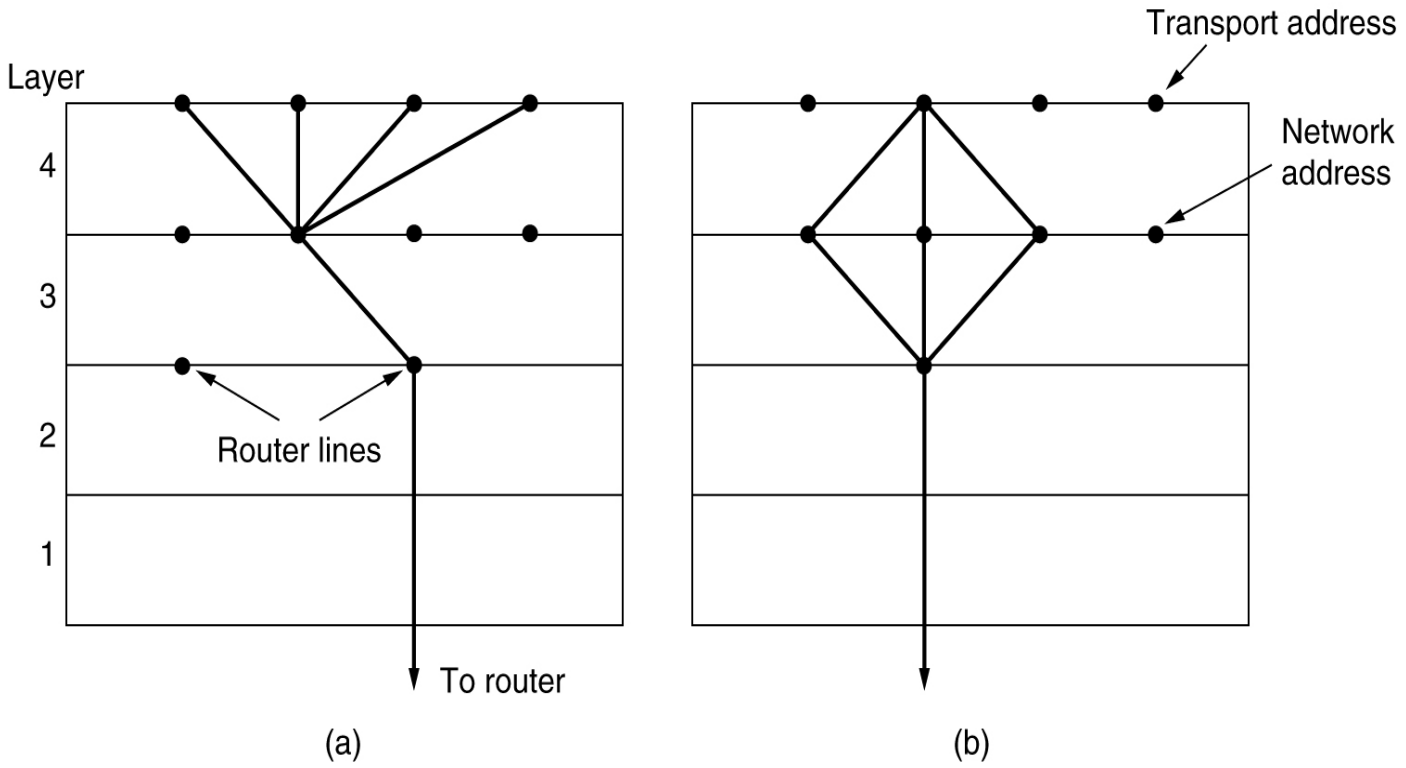
流量控制和缓冲策略(2)

- 动态缓冲区管理窗口
  - 16 行潜在的死锁；定期在连接上发送控制段。

	A	Message	B	Comments
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	• • •	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	• • •	<ack = 6, buf = 4>	←	Potential deadlock

6.2.5 多路复用

- 向上多路复用：多个传输连接使用同一网络连接
- 向下多路复用：一个传输连接使用多个网络连接



6.2.6 崩溃恢复

- 发送主机状态
  - S0：没有未被确认的TPUD
  - S1：有一个未被确认的TPUD
- 接收主机策略
  - 先确认后写：AC(W)、AWC、C(AW)
  - 先写后确认：WC(A)、WAC、C(WA)
- 发送主机策略
  - 总是重发
  - 从不重发
  - S0状态下重发
  - S1状态下重发
- 结论
  - 图6-18（P408）
  - 崩溃恢复只能由上一层完成

Strategy used by receiving host

Strategy used by sending host	First ACK, then write			First write, then ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit	OK	DUP	OK	OK	DUP	DUP
Never retransmit	LOST	OK	LOST	LOST	OK	OK
Retransmit in S0	OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1	LOST	OK	OK	OK	OK	DUP

OK = Protocol functions correctly
DUP = Protocol generates a duplicate message
LOST = Protocol loses a message

6.3 拥塞控制

- 理想的带宽分配
- 调整发送速率
- 无线问题

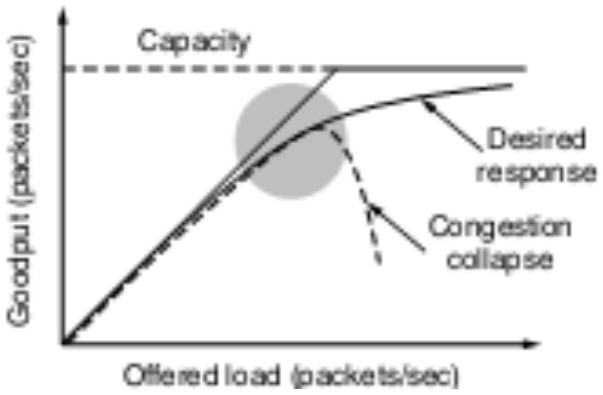
理想的带宽分配

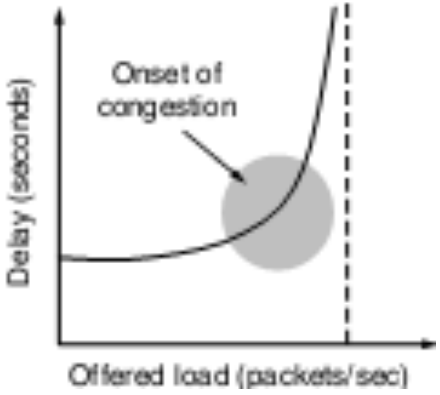
- 效率与功率

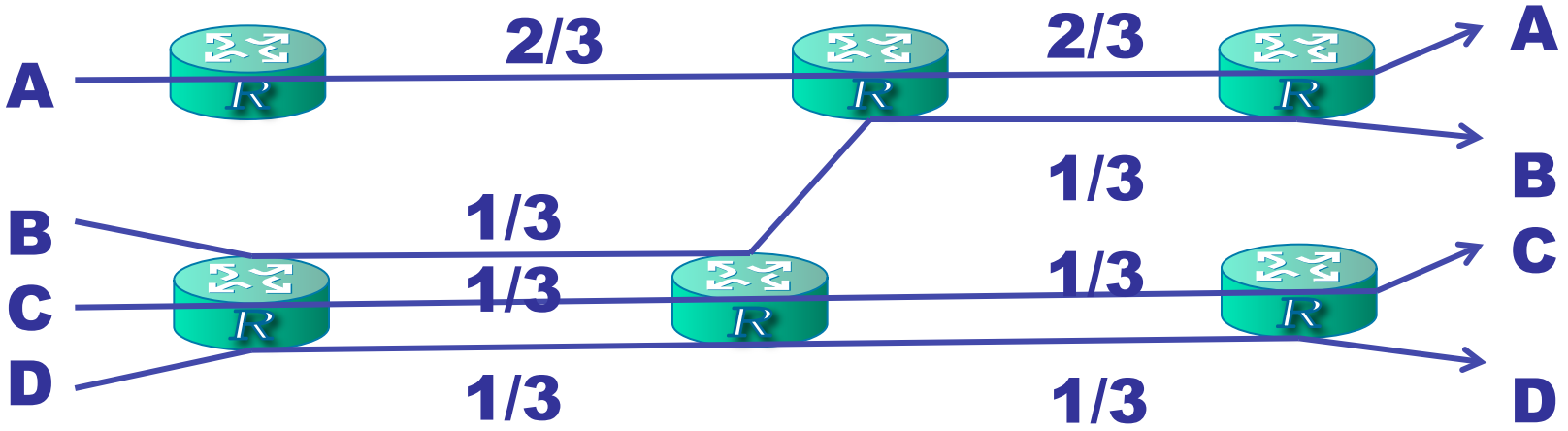
  功率=负载/延迟

- 最大-最小公平性

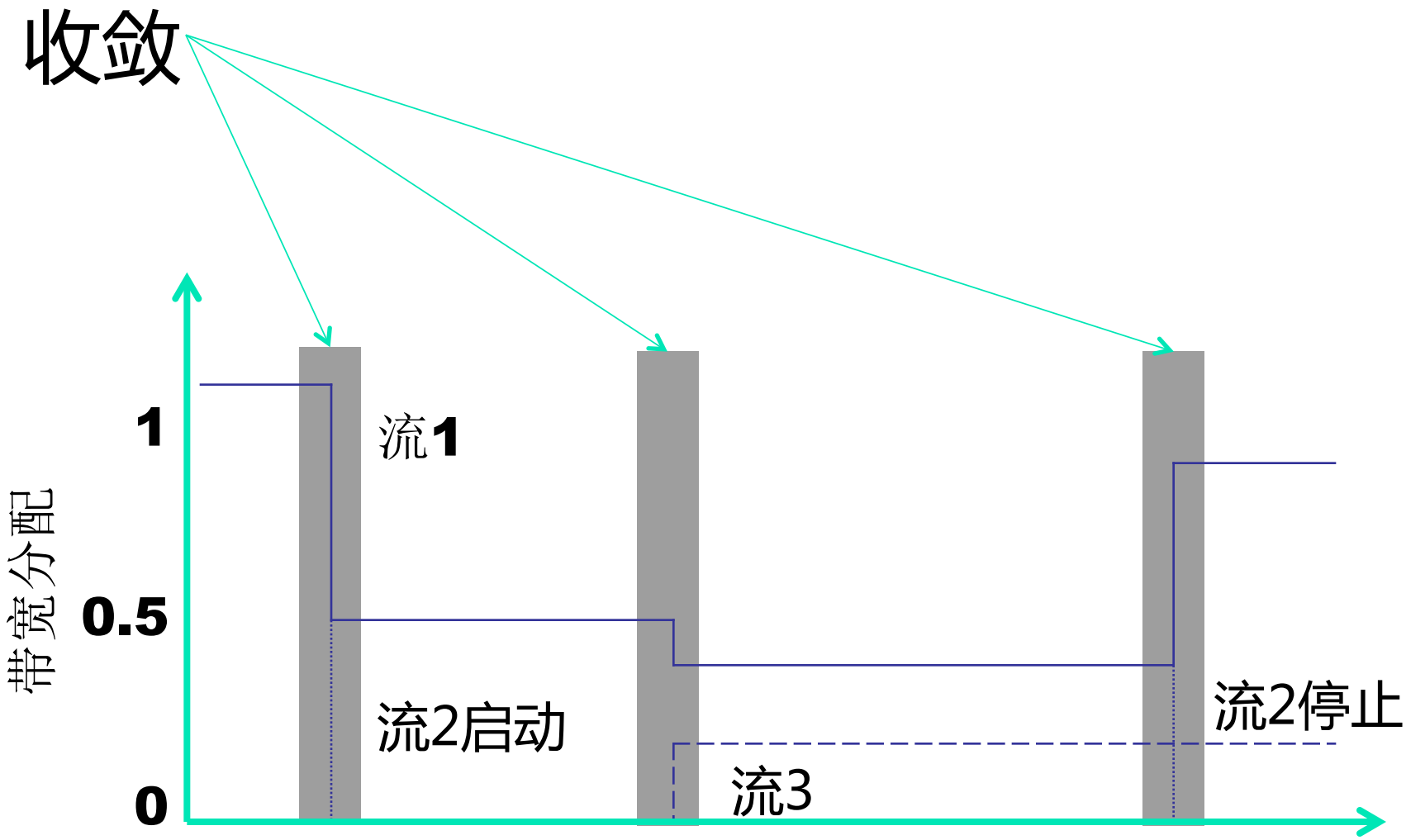
  ▪均等带宽比例





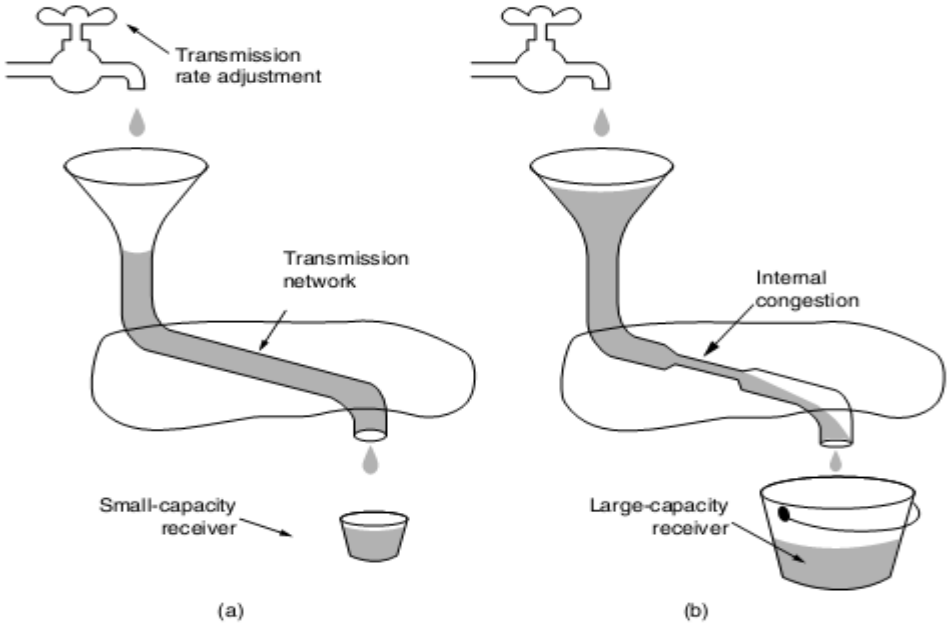


快速收敛



调整发送速率

- 显式拥塞协议



协议	信号	显式否	精确否
XCP	使用速率	是	是
TCP	拥塞警告	是	否
FAST TCP	端到端延迟	否	是
Compound TCP	数据包丢失&端到端延迟	否	是
CUBIC TCP	数据包丢失	否	否
TCP	数据包丢失	否	否

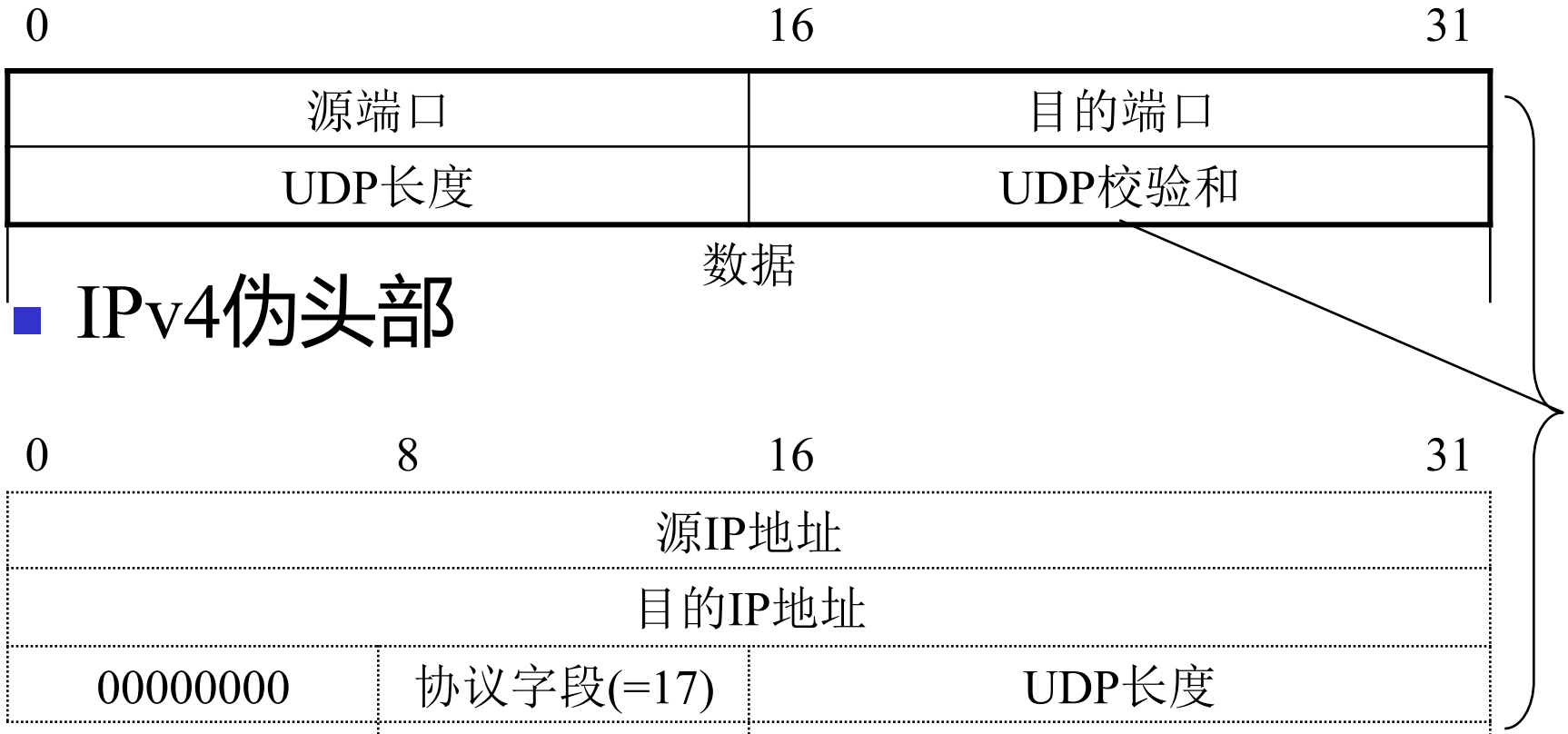
- 加法递增乘法递减(AIMD)

6.4 因特网传输协议UDP

- UDP介绍
- 远过程调用
- 实时传输协议

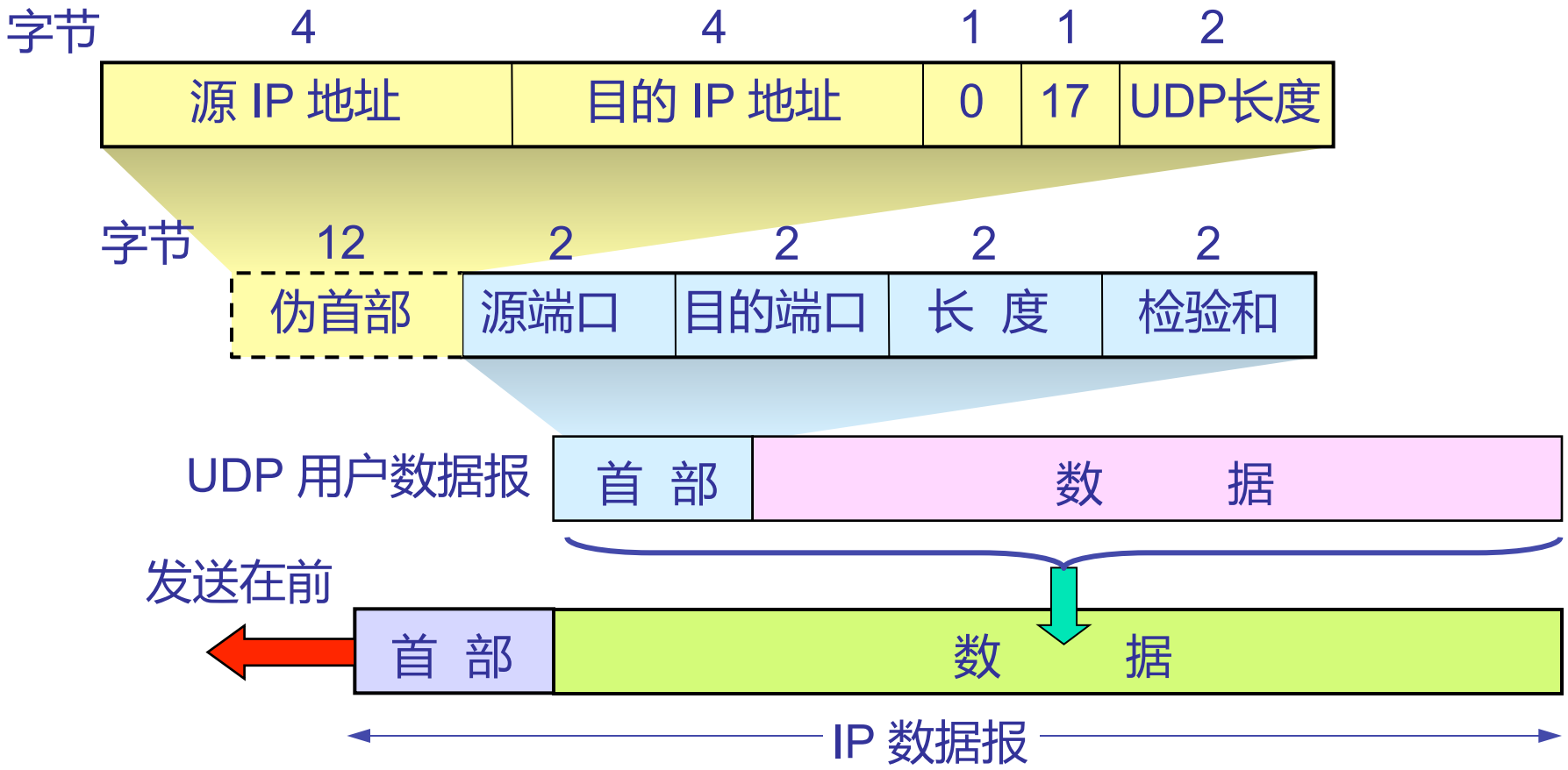
6.4.1 UDP介绍

- 用户数据报协议
- UDP头部



- IPv4伪头部

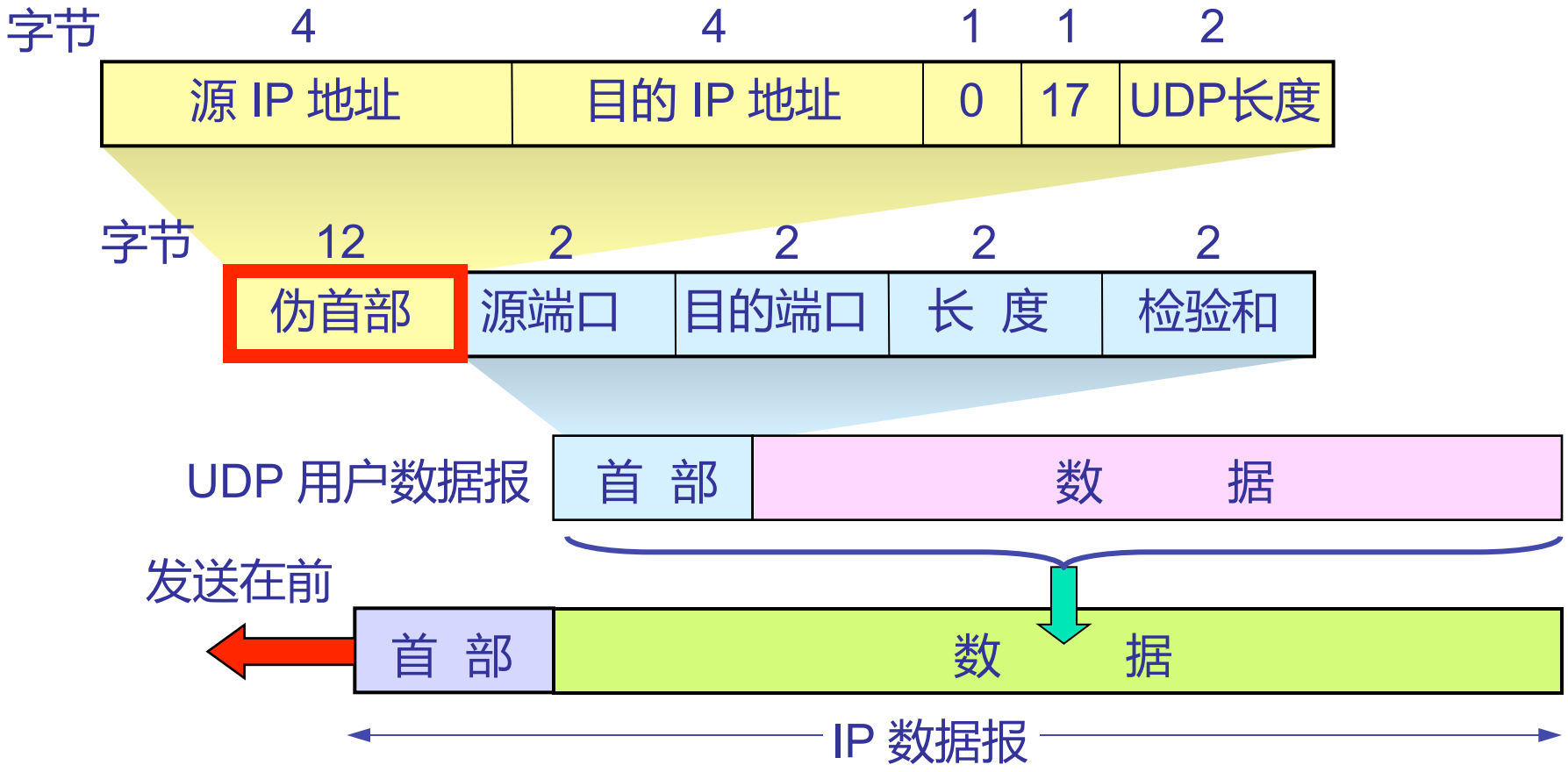
UDP 用户数据报的首部格式



用户数据报 UDP 有两个字段：数据字段和首部字段。首部字段有 8 个字节，由 4 个字段组成，每个字段都是两个字节。

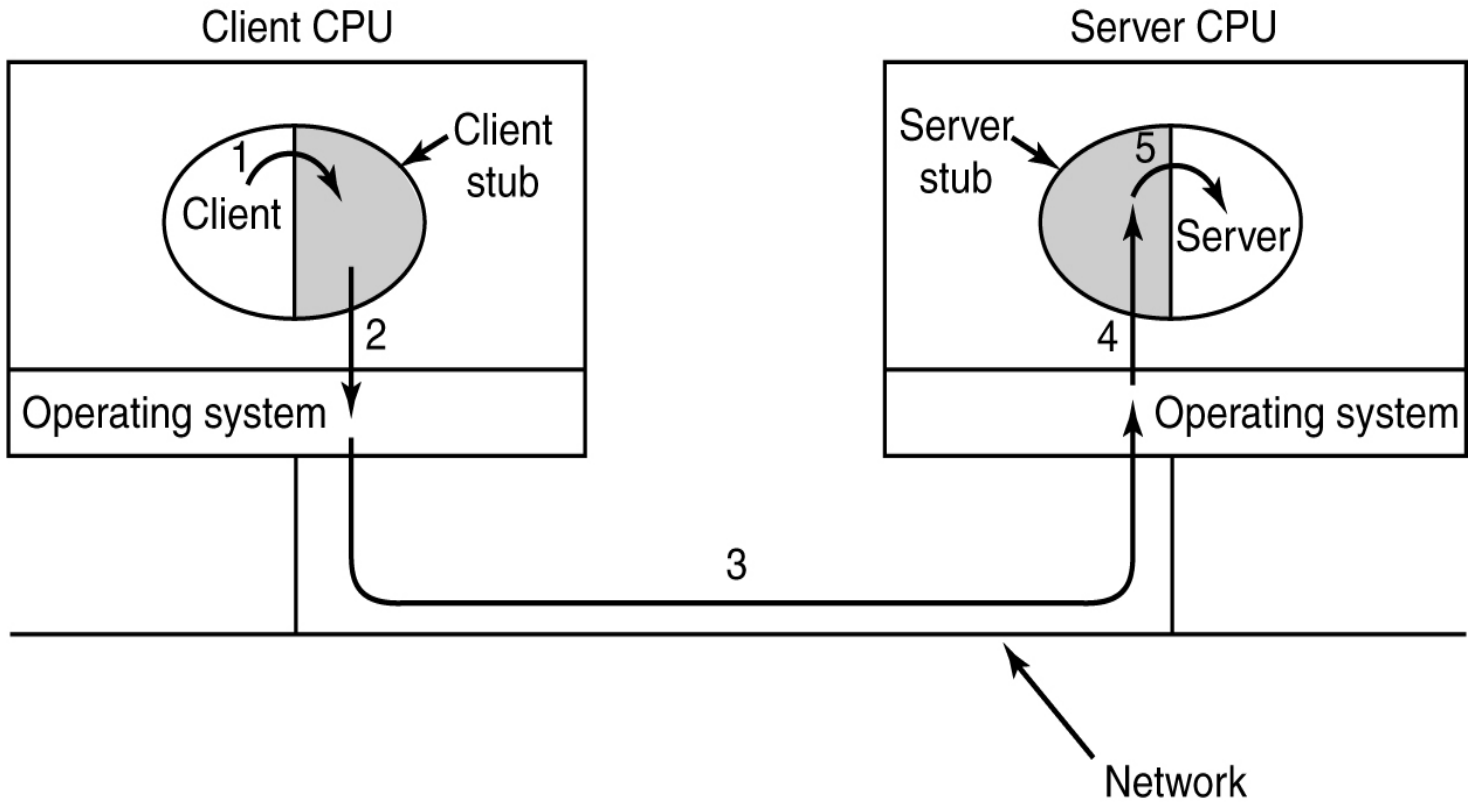


在计算检验和时，临时把“伪首部”和 UDP 用户数据报连接在一起。伪首部仅仅是为了计算检验和。



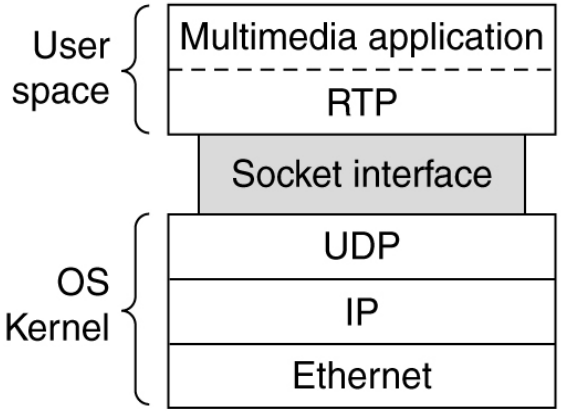
6.4.2 远过程调用

- RPC
  - 客户存根：封装参数（列集）
  - 服务器存根：解封参数（散集）

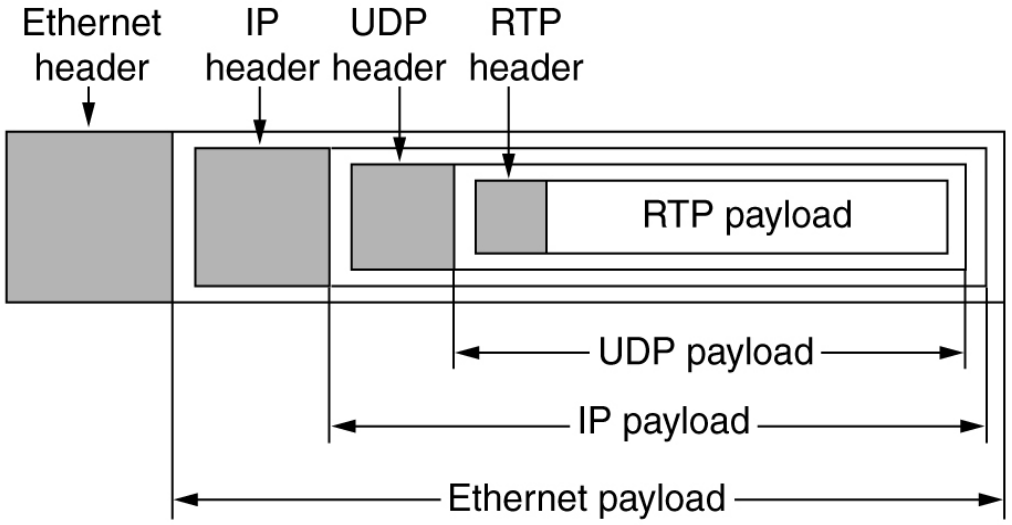


6.4.3 实时传输协议

- RTP(RFC3550)—广泛应用于多媒体应用程序
- RTP协议
  - 以数据包的形式传输音频和视频数据
- 接收端
  - 在正确的时间播放音频和视频



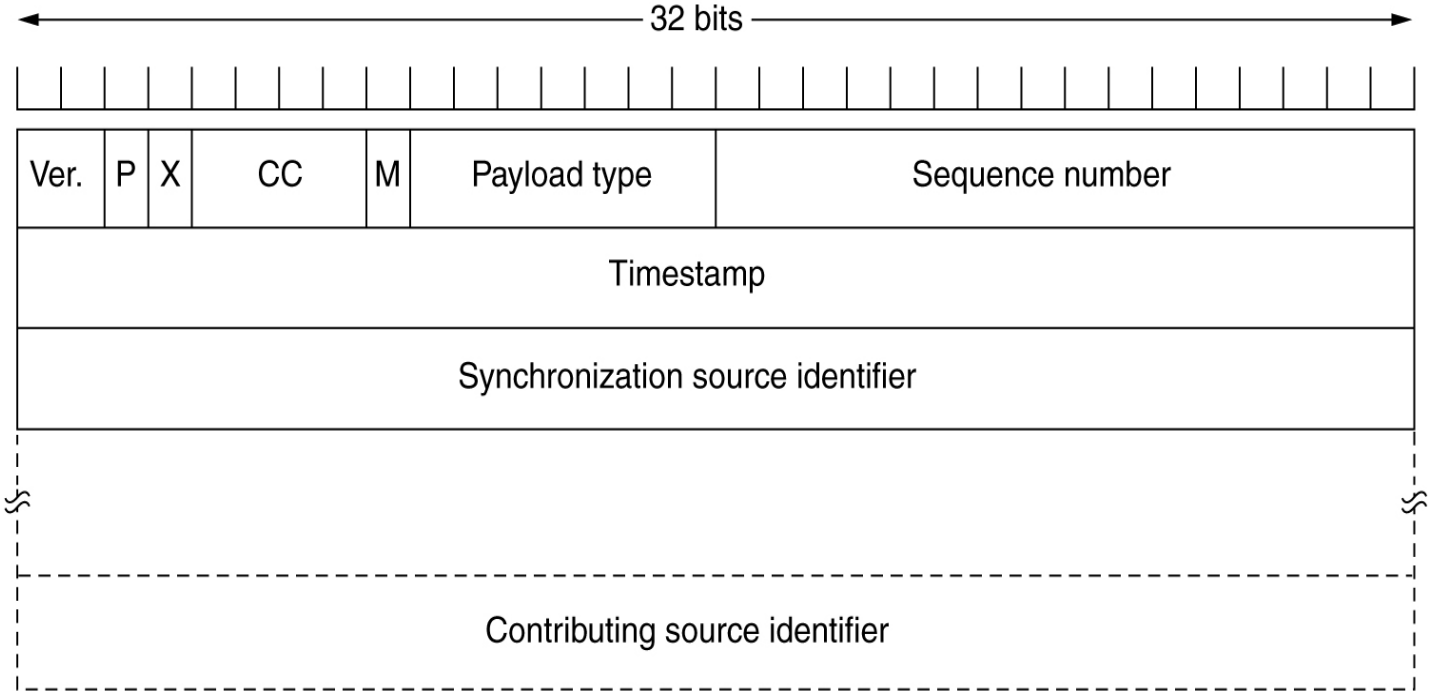
(a)



(b)

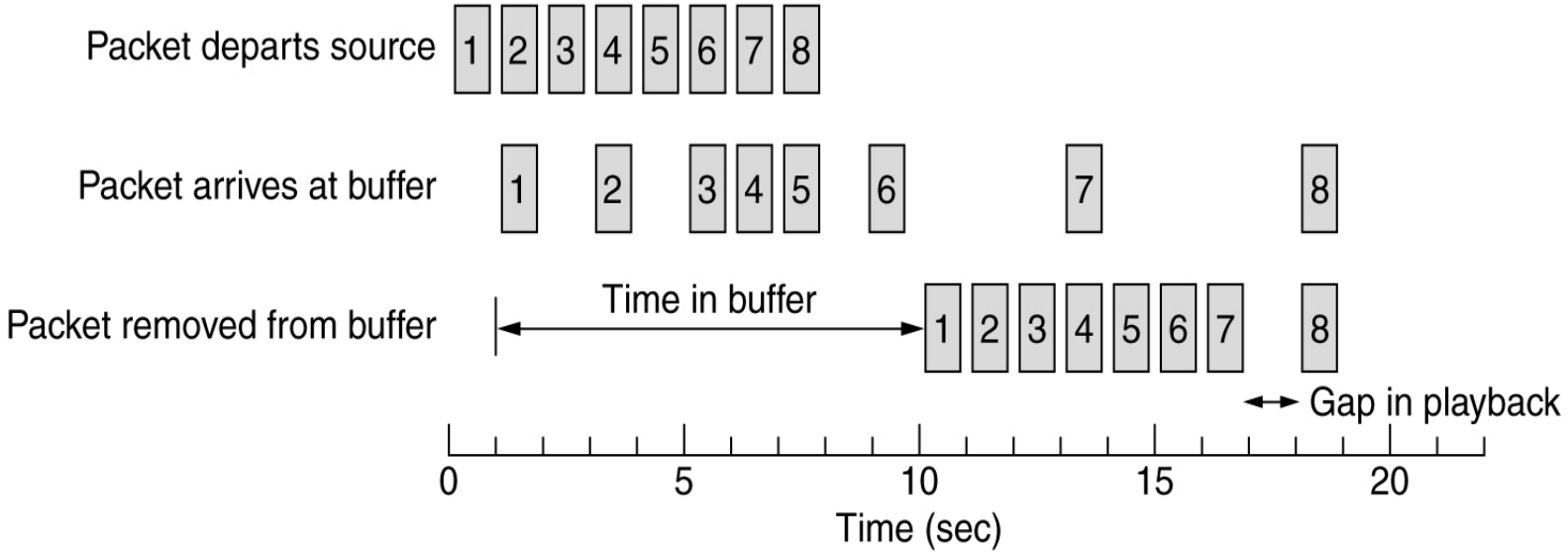
实时传输协议(2)

- RTP——是一个在应用层上实现的传输协议
  - 将多个实时数据流复用到一个UDP数据段中
  - 时间戳：网络延迟变化，多个流之间同步
- RTP头



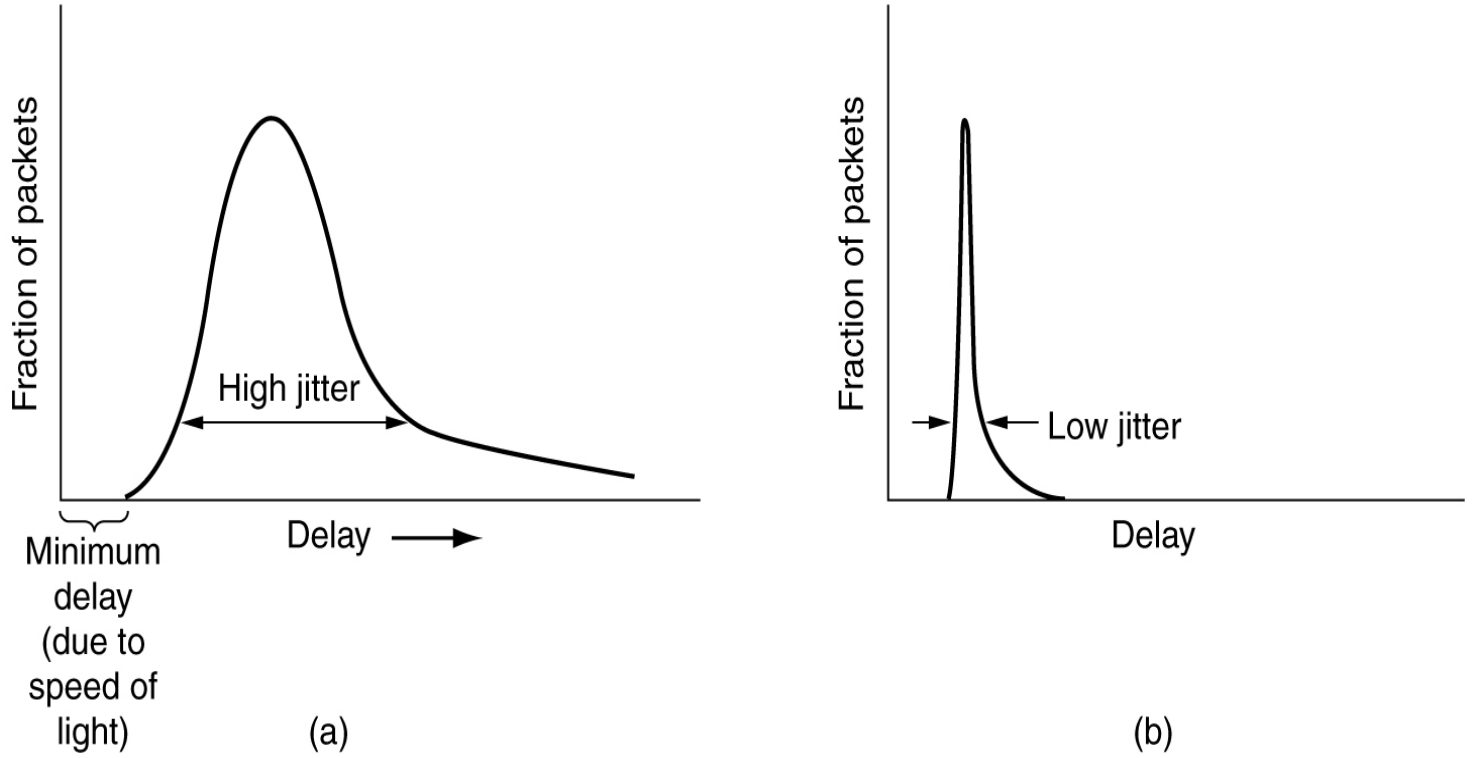
实时传输协议(3)

- RTCP——实时传输控制协议
  - 向源端提供网络特征反馈信息
  - 延迟、抖动、带宽、拥塞等
- 带有缓冲和抖动控制的播放



抖动控制

- 播放点
  - 最小延迟、平均延迟
  - 等待时间：高抖动、低抖动



6.5 因特网传输协议TCP

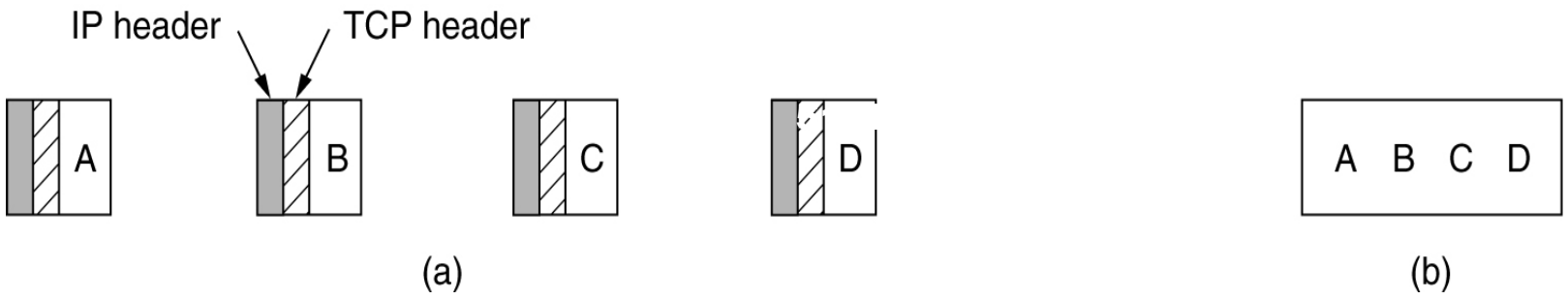
- TCP概述
- TCP服务模型
- TCP协议
- TCP段的头
- TCP连接建立
- TCP连接释放
- TCP连接管理模型
- TCP滑动窗口
- TCP计时器管理
- TCP拥塞控制
- TCP未来

6.5.1 TCP概述

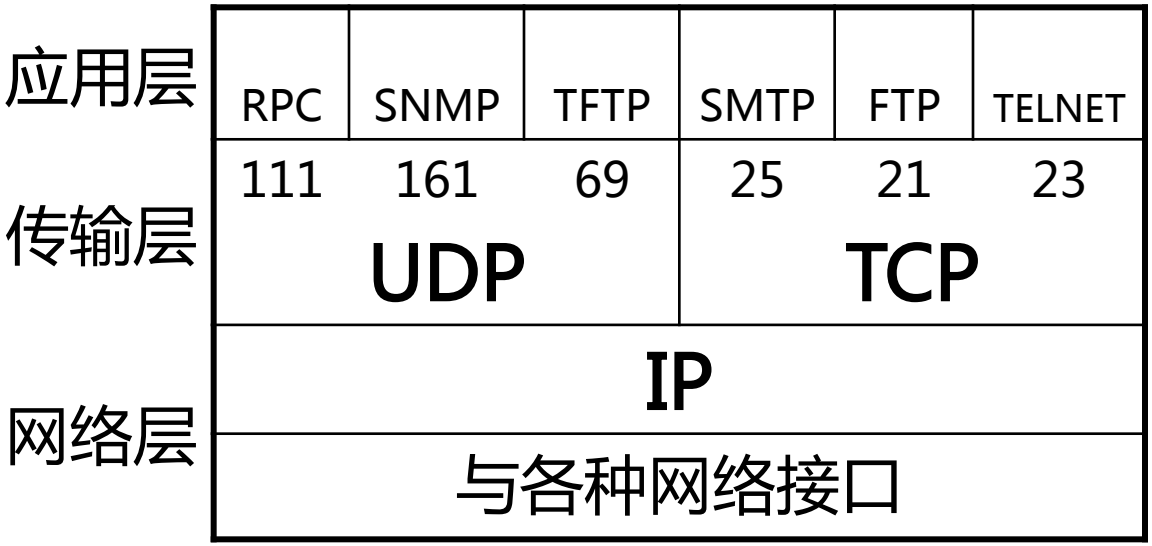
- 传输控制协议TCP（Transmission Control Protocol）
  - 面向连接的、可靠的、端到端的、基于字节流的传输协议；
  - RFC 793+，1122，1323，2018，2581，2873，2988等。
- 在不可靠的互联网上提供可靠的端到端服务
- TCP实体
  - 一个库过程
  - 一个用户进程
  - 内核的一部分

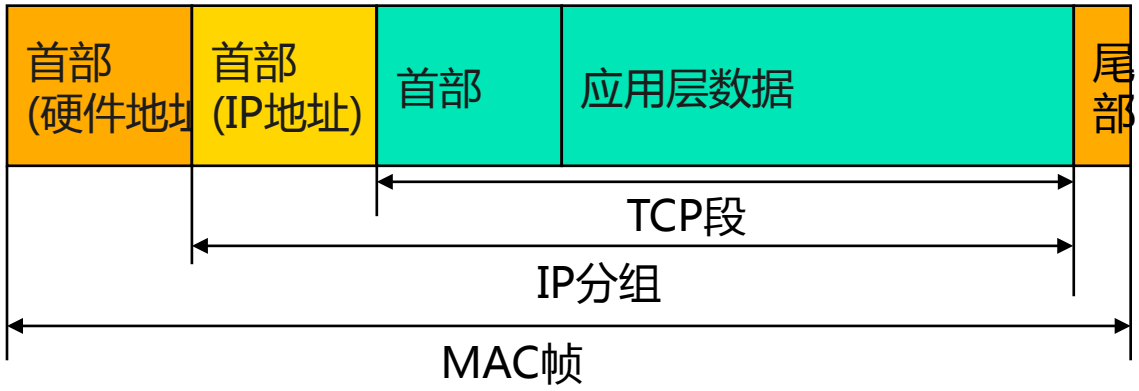
6.5.2 TCP服务模型

- TCP服务由发送端和接收端创建一种称为套接字（socket）的端点获得。
- 套接字: 主机ip地址+16位数值的端口号
- 一个套接字可用于多个连接
- TCP连接都是全双工的，且是点到点的
  - 是字节流，不是消息流



TCP服务模型(2)

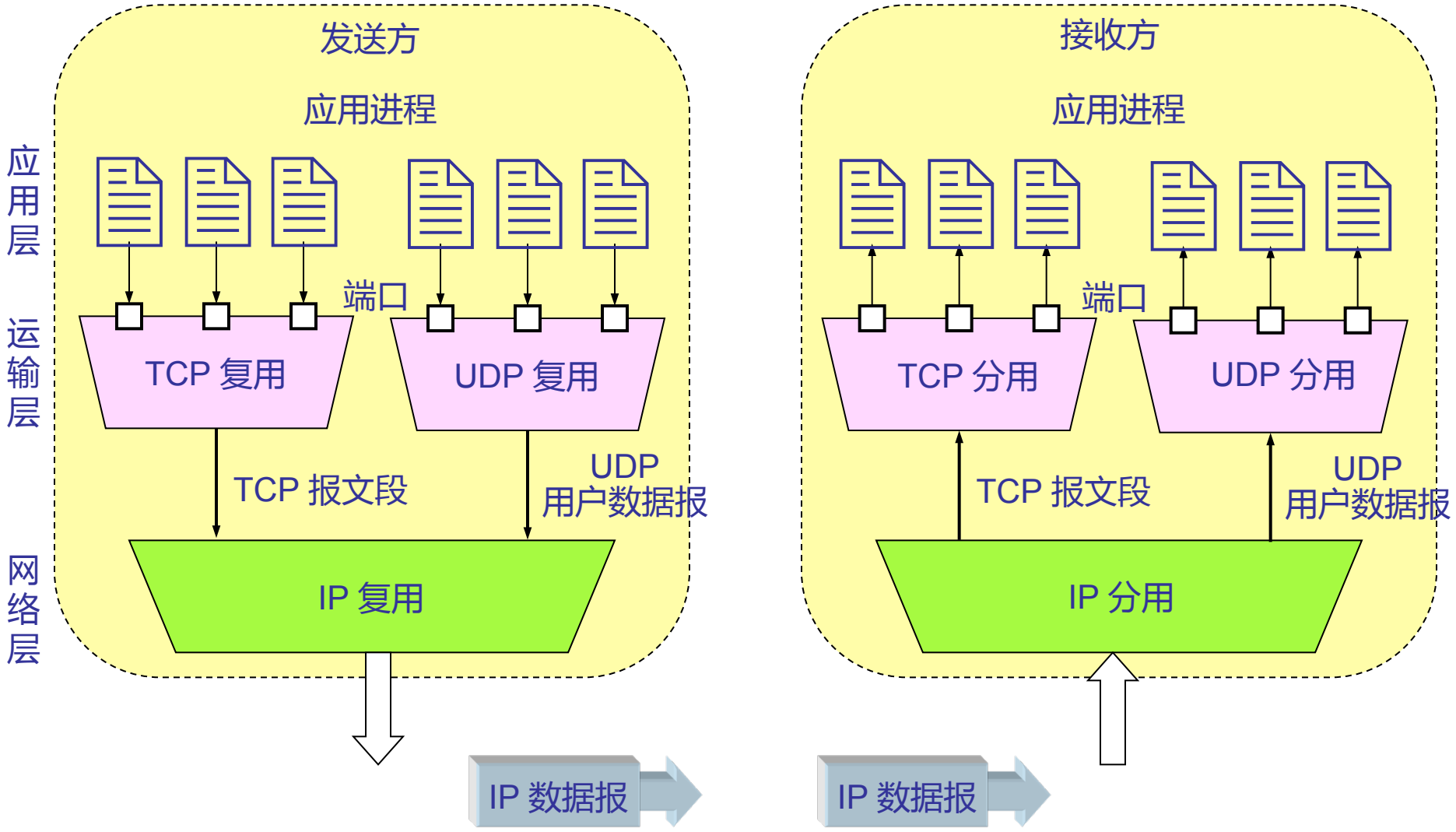

应用层
传输层
网络层
RPC
SNMP
TFTP
SMTP
FTP
TELNET
111
161
69
25
21
23
UDP
TCP
IP
与各种网络接口


首部
(硬件地址)
首部
(IP地址)
首部
应用层数据
尾部
TCP段
IP分组
MAC帧

端口的概念

- 端口就是传输层服务访问点 TSAP。
- 端口的作用就是让应用层的各种应用进程都能将其数据通过端口向下交付给传输层，以及让传输层知道应当将其报文段中的数据向上通过端口交付给应用层相应的进程。
- 从这个意义上讲，端口是用来标志应用层的进程。

端口在进程之间的通信中所起的作用

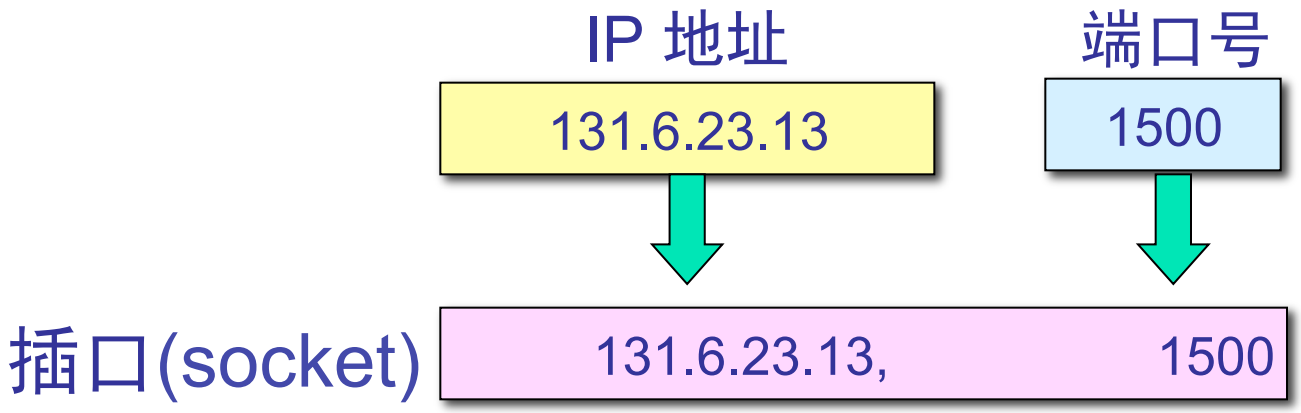


端口标志

- 端口用一个 16 bit 端口号进行标志。
- 端口号只具有本地意义，即端口号只是为了标志本计算机应用层中的各进程。在因特网中不同计算机的相同端口号是没有联系的。
- 有两类端口
  - 一类是熟知端口，其数值一般为 0~1023。当一种新的应用程序出现时，必须为它指派一个熟知端口。
  - 另一类则是一般端口，用来随时分配给请求通信的客户进程。

插口(socket)

- TCP 使用“连接”(而不仅仅是“端口”)作为最基本的抽象，同时将TCP连接的端点称为插口(socket)，或套接字、套接口。
- 插口和端口、IP 地址的关系是：



同一个名词 socket有多种不同的意思

- 应用编程接口API称为 socket API, 简称为 socket。
- socket API 中使用的一个函数名也叫作socket。
- 调用 socket 函数的端点称为 socket。
- 调用 socke t函数时其返回值称为 socket描述符，可简称为 socket。
- 在操作系统内核中连网协议的 Berkeley 实现，称为 socket 实现。

6.5.3 TCP协议

- 每个字节都有自己独有的32位序号
- 以段形式交换数据
- TCP段
  - 20字节的头
  - IP有效载荷：65515B
  - 网络最大传输单元（MTU）
  - 某条链路有更小MTU时——路径MTU发现
    - 利用ICMP错误消息发现某条路径上任意一条链路的最小MTU。
- 使用动态窗口大小的滑动窗口协议

6.5.4 TCP段的头

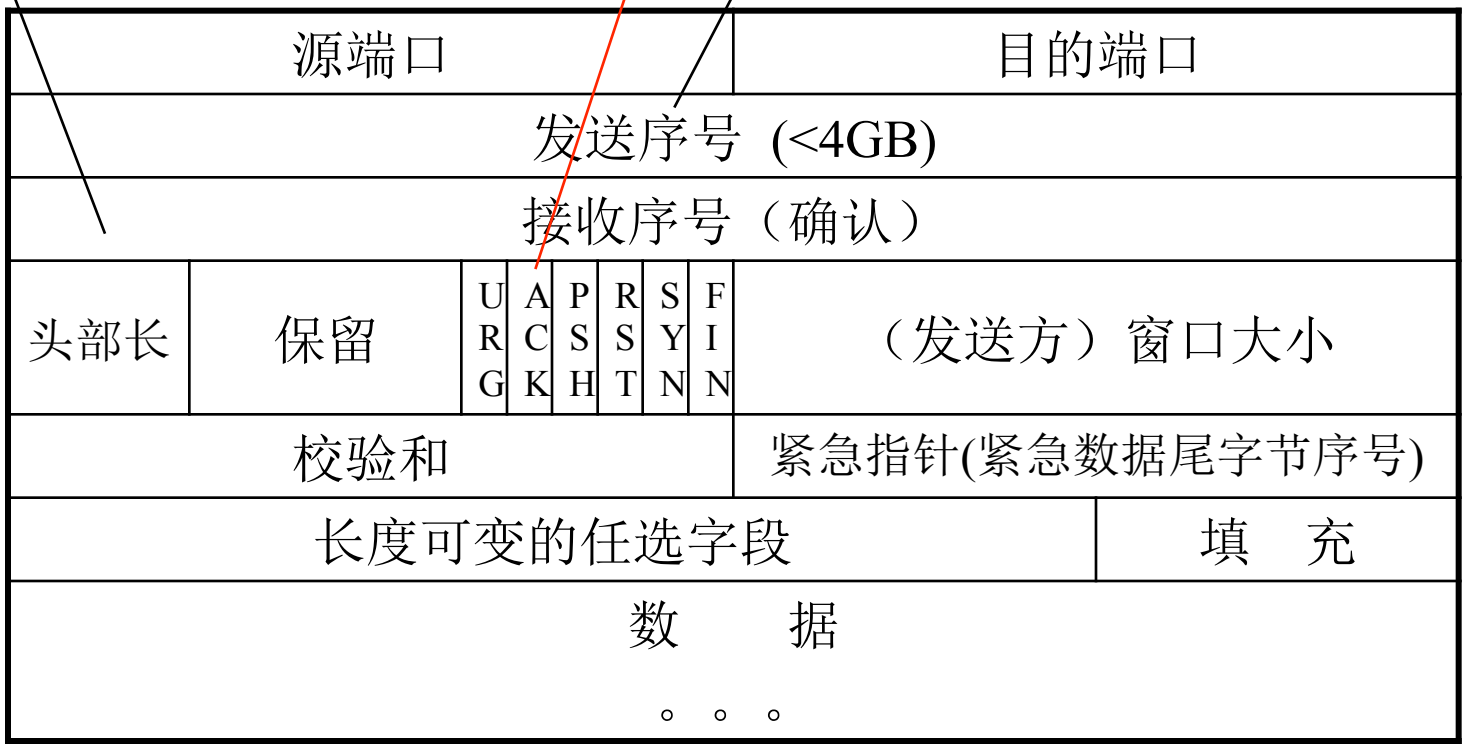
- TCP段的结构
  - 净载荷

    65535-20-20=65495

紧急、确认、急迫、重建、同步、终止

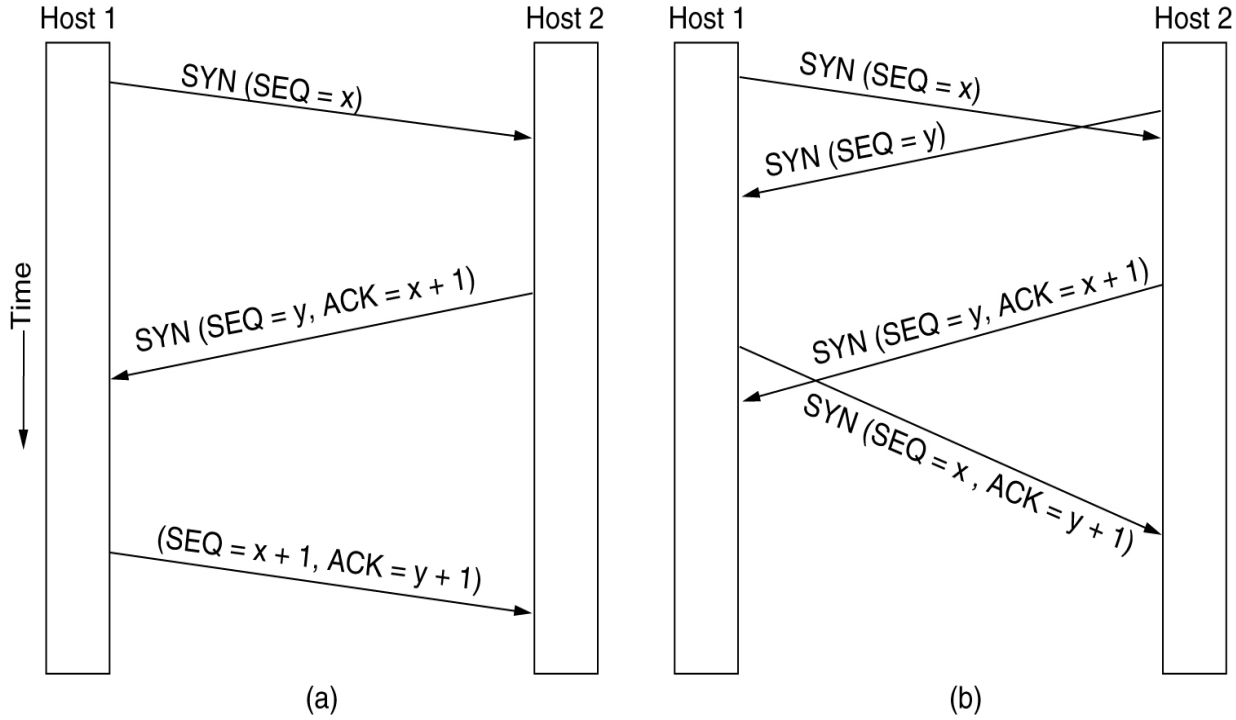
本报文段数据部分首字节在报文中的序号

5~15(W)
(W=32b)



6.5.5 TCP连接建立

- 建立连接采用三次握手方法



- SYN泛洪攻击
  - 恶意用户发送SYN连接请求，且故意不做连接建立

6.5.6 TCP连接释放

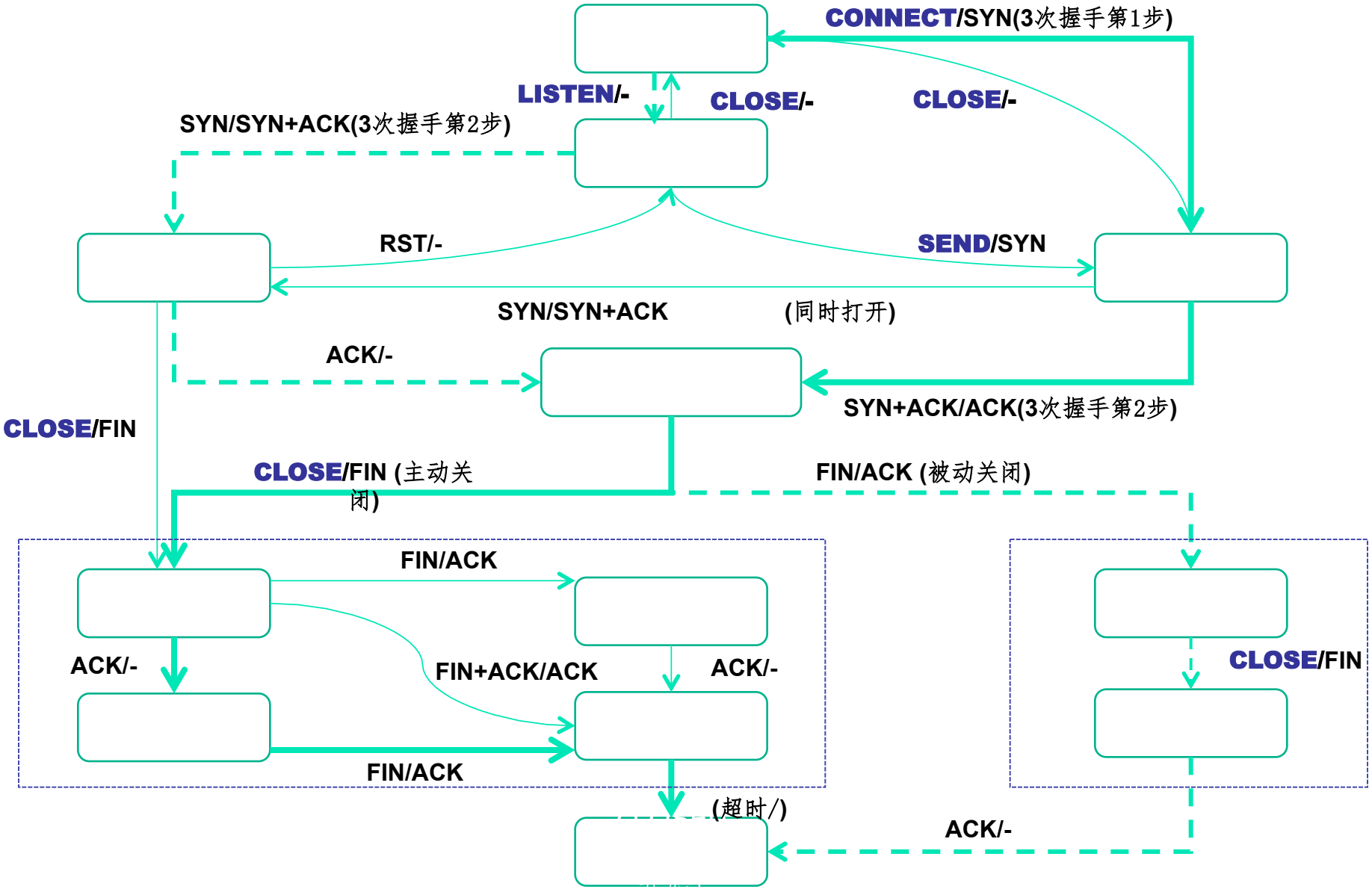
- 释放连接
  - 发送设置了FIN标志位的TCP段
  - 三个TCP段
    - FIN
    - ACK+FIN
    - ACK

6.5.7 TCP连接管理模型

- TCP连接管理有限自动机状态

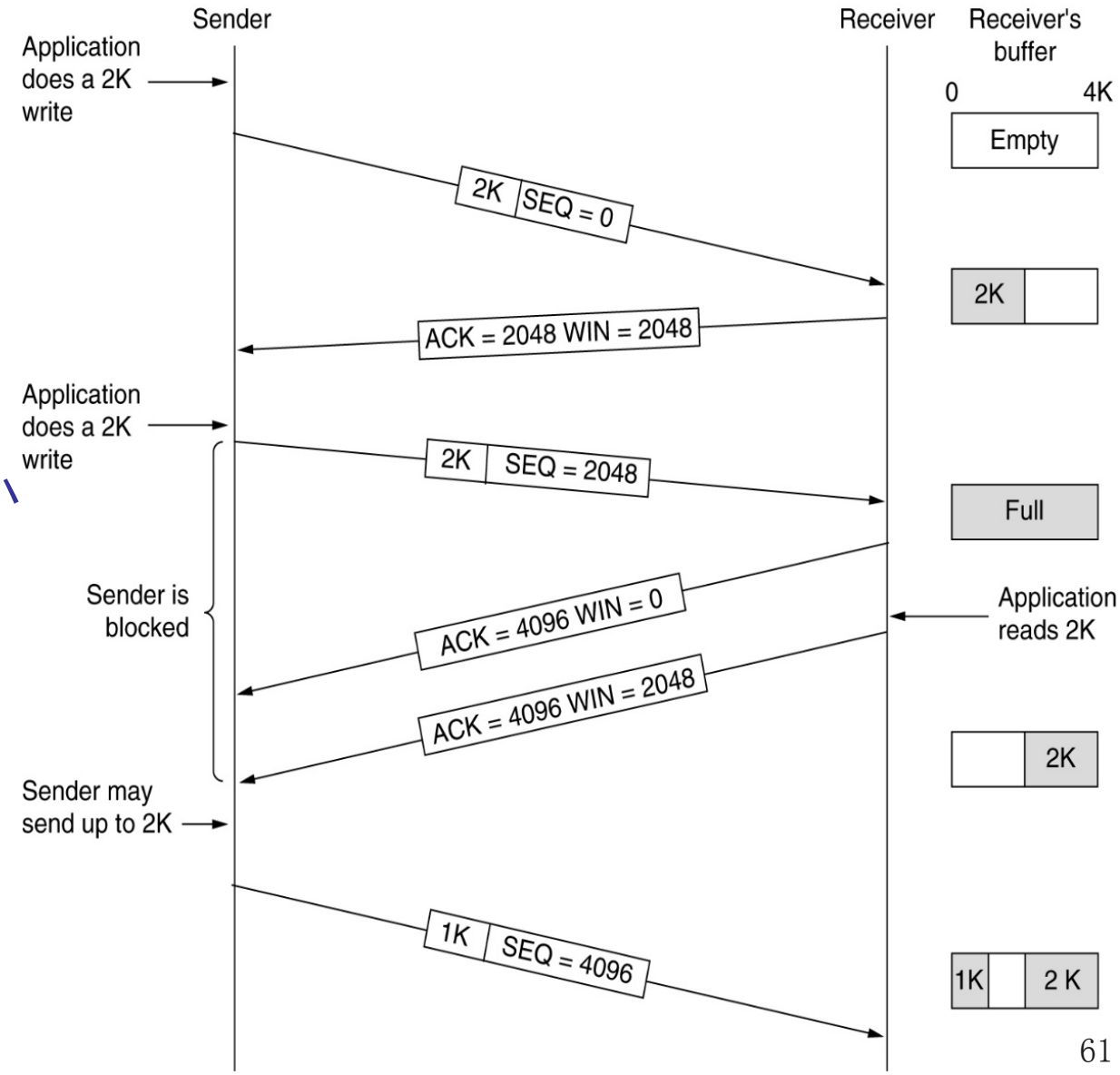
State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

TCP连接管理有限自动机



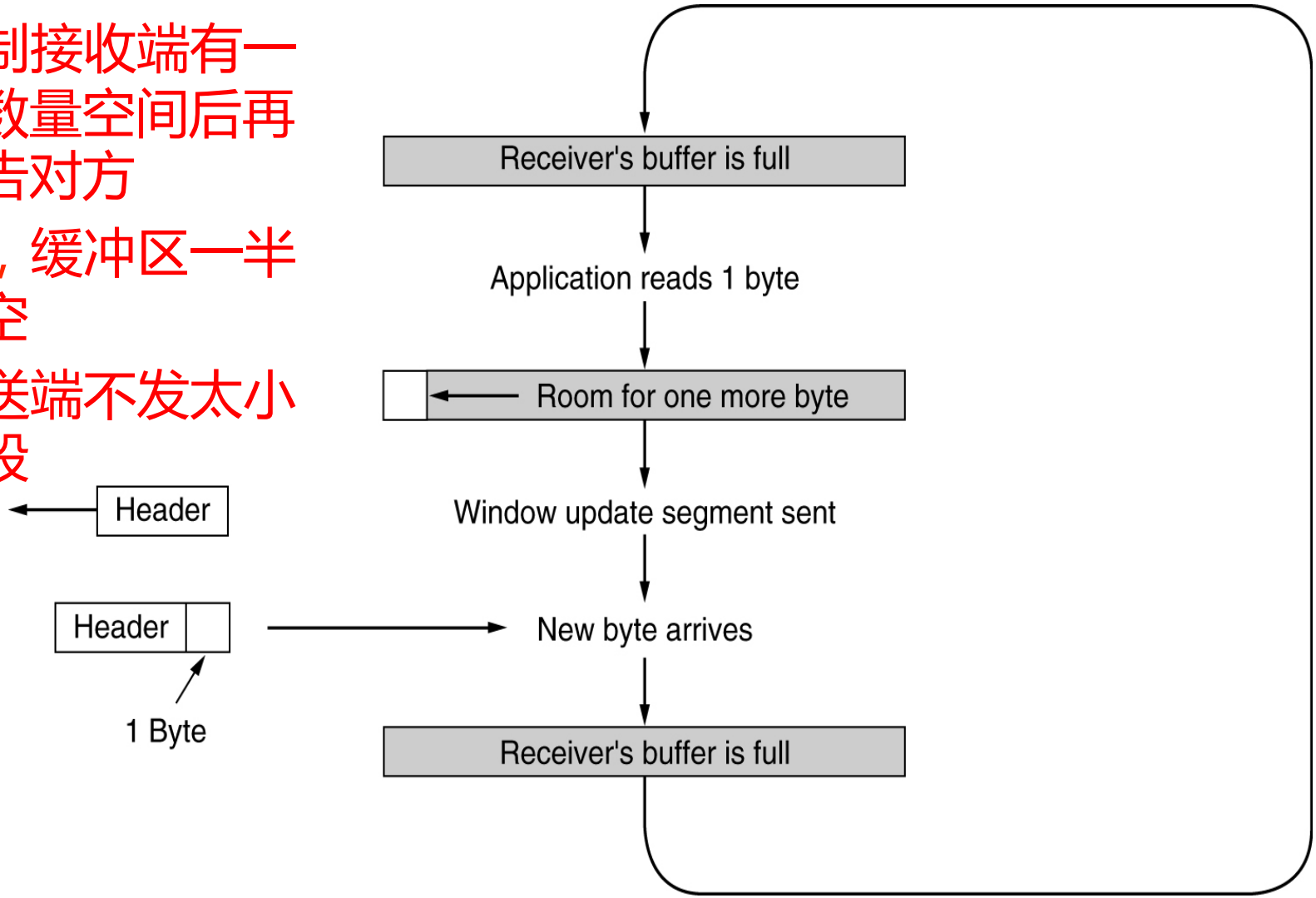
6.5.8 TCP滑动窗口

- TCP滑动窗口
  - 接收端4096B
- 窗口为0
  - 紧急数据可发送
  - 窗口探测
    - 强制宣告窗口大小



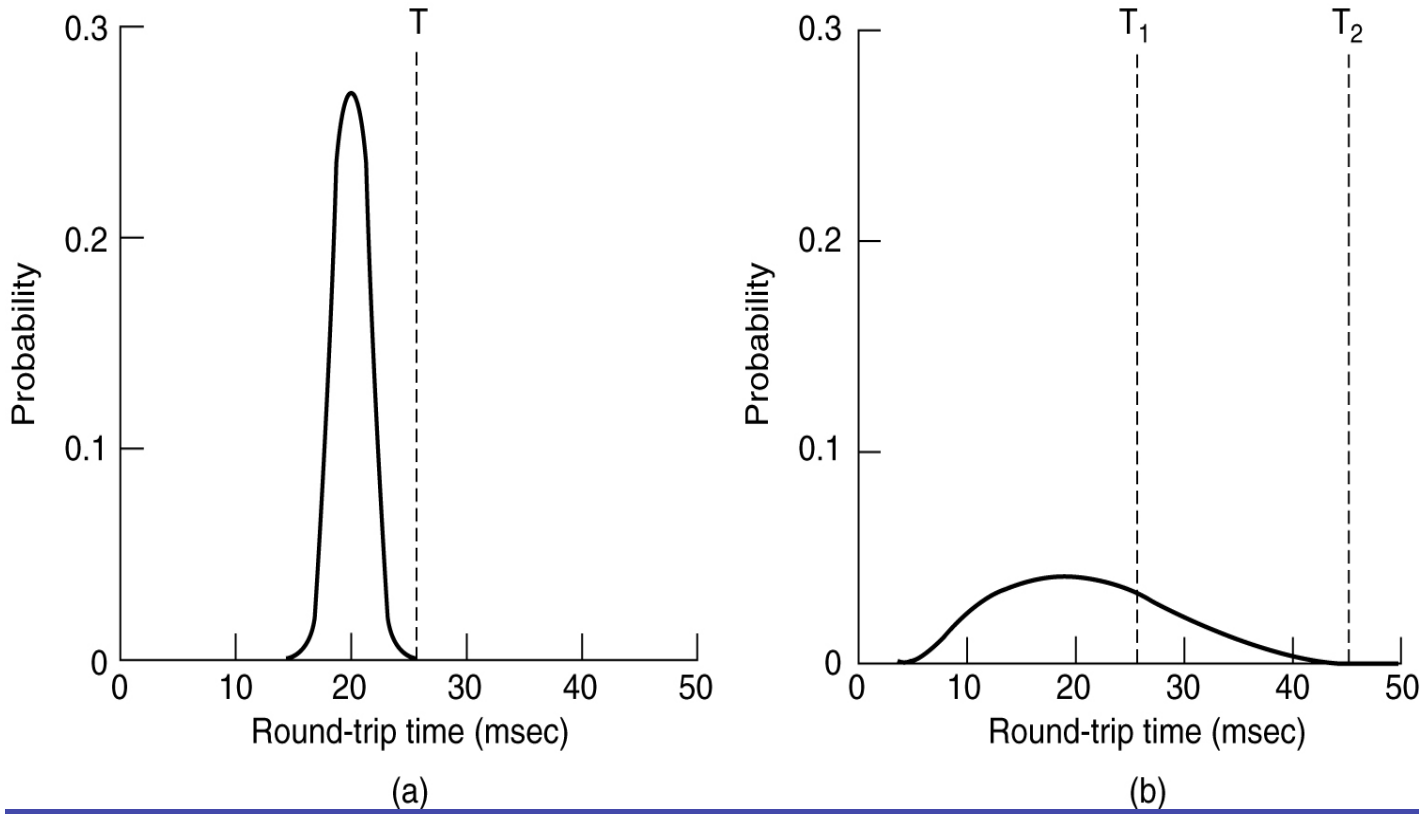
低能窗口

- 解决方案
  - 强制接收端有一定数量空间后再通告对方
  - 如，缓冲区一半为空
  - 发送端不发太小的段



6.5.9 TCP计时器管理

- 重传计时器



数据链路层的确认到达时间概率密度　　TCP的确认到达时间的概率密度

TCP计时器管理(2)

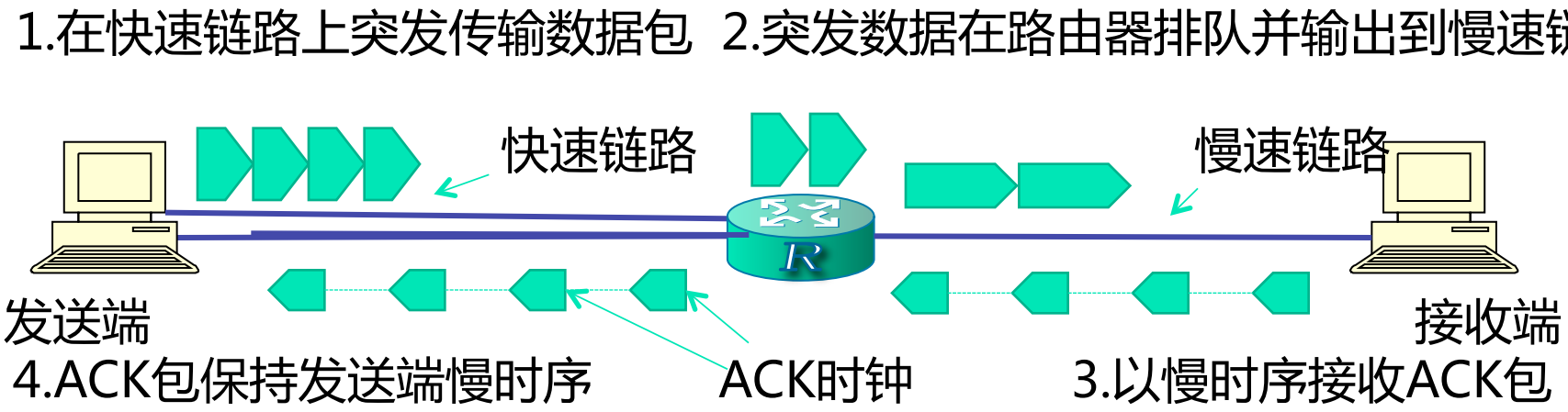
- 重传定时器算法
  - 平滑往返时间SRTT

    $SRTT = \alpha \times SRTT + (1-\alpha)R$ （一般$\alpha=7/8$）

    本次测量确认所花时间R
  - 超时值
    - 原来 = 2×RTT
    - 现在：往返时间变化RTTVAR
      - $RTTVAR = \beta \times RTTVAR + (1-\beta) \mid SRTT - R \mid$
      - 超时值: $RTO = SRTT + 4 \times RTTVAR$
- 持续计时器：探询接收方，返回窗口大小
- 保活计时器

6.5.10 TCP拥塞控制

- TCP作用
  - 在拥塞控制和可靠传输中发挥主要作用
- 拥塞窗口
  - 采用AIMD(加增乘减)规则调整窗口大小
  - 发送窗口 = min(拥塞窗口，接收窗口)
- 确认时钟(ACK时钟)



慢启动和拥塞避免

- 发送端的主机在确定发送报文段的速率时，既要根据接收端的接收能力，又要从全局考虑不要使网络发生拥塞。
- 因此，每一个 TCP 连接需要有以下两个状态变量：
  - 接收端窗口 rwnd (receiver window) 又称为通知窗口(advertised window)。
  - 拥塞窗口 cwnd (congestion window)。

接收端窗口 rwnd 和拥塞窗口 cwnd

(1) 接收端窗口 rwnd ，这是接收端根据其目前的接收缓存大小所许诺的最新的窗口值，是来自接收端的流量控制。接收端将此窗口值放在 TCP 报文的首部中的窗口字段，传送给发送端。

(2) 拥塞窗口 cwnd (congestion window) 是发送端根据自己估计的网络拥塞程度而设置的窗口值，是来自发送端的流量控制。

发送窗口的上限值

- 发送端的发送窗口的上限值应当取为接收端窗口 rwnd 和拥塞窗口 cwnd 这两个变量中较小的一个，即应按以下公式确定：

> 发送窗口的上限值 = Min [rwnd, cwnd]

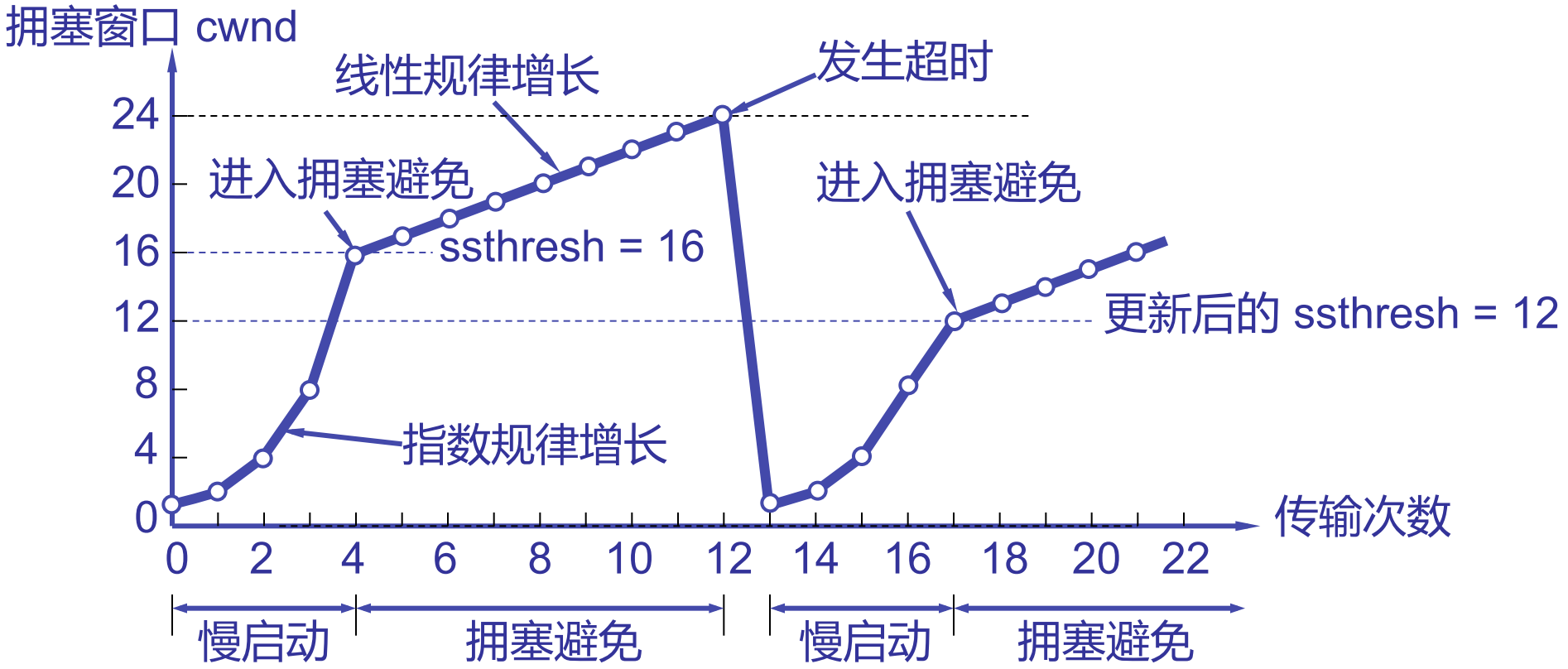
当 rwnd < cwnd 时，是接收端的接收能力限制发送窗口的最大值。

- 当 cwnd < rwnd 时，则是网络的拥塞限制发送窗口的最大值。

慢启动算法的原理

- 在主机刚刚开始发送报文段时可先将拥塞窗口 cwnd 设置为一个最大报文段 MSS 的数值。
- 在每收到一个对新的报文段的确认后，将拥塞窗口增加至多一个 MSS 的数值。
- 用这样的方法逐步增大发送端的拥塞窗口 cwnd，可以使分组注入到网络的速率更加合理。

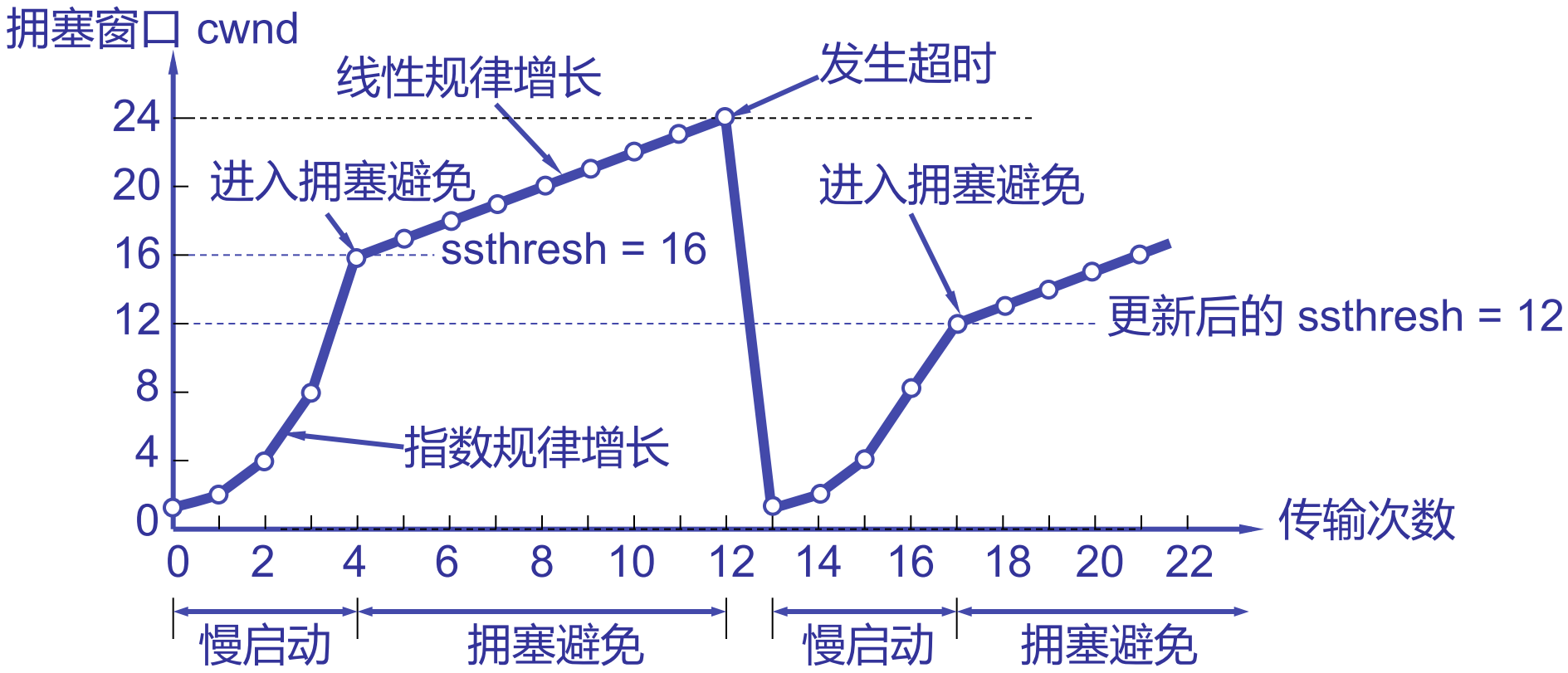
慢启动和拥塞避免算法的实现举例



当 TCP 连接进行初始化时，将拥塞窗口置为 1。图中的窗口单位不使用字节而使用报文段。

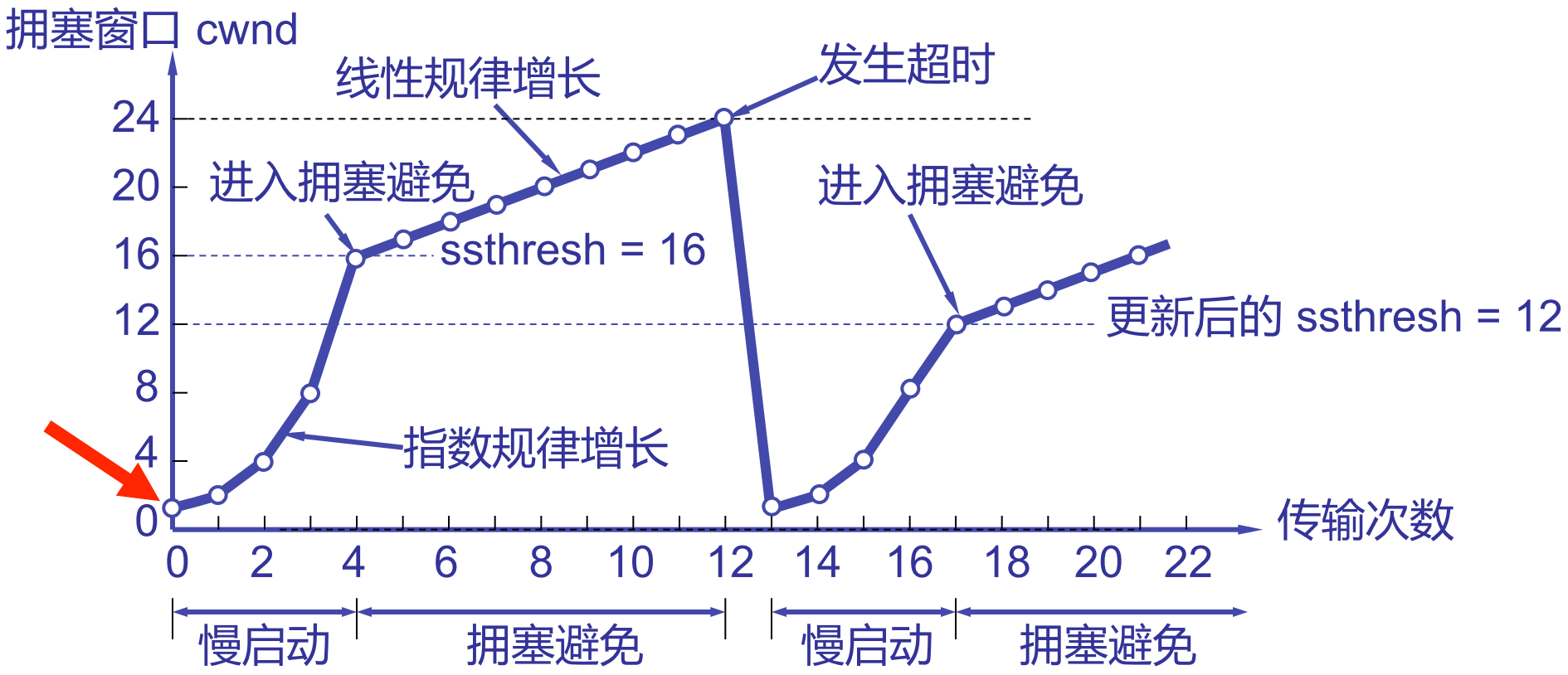
慢启动门限的初始值设置为16个报文段,即ssthresh=16

慢启动和拥塞避免算法的实现举例



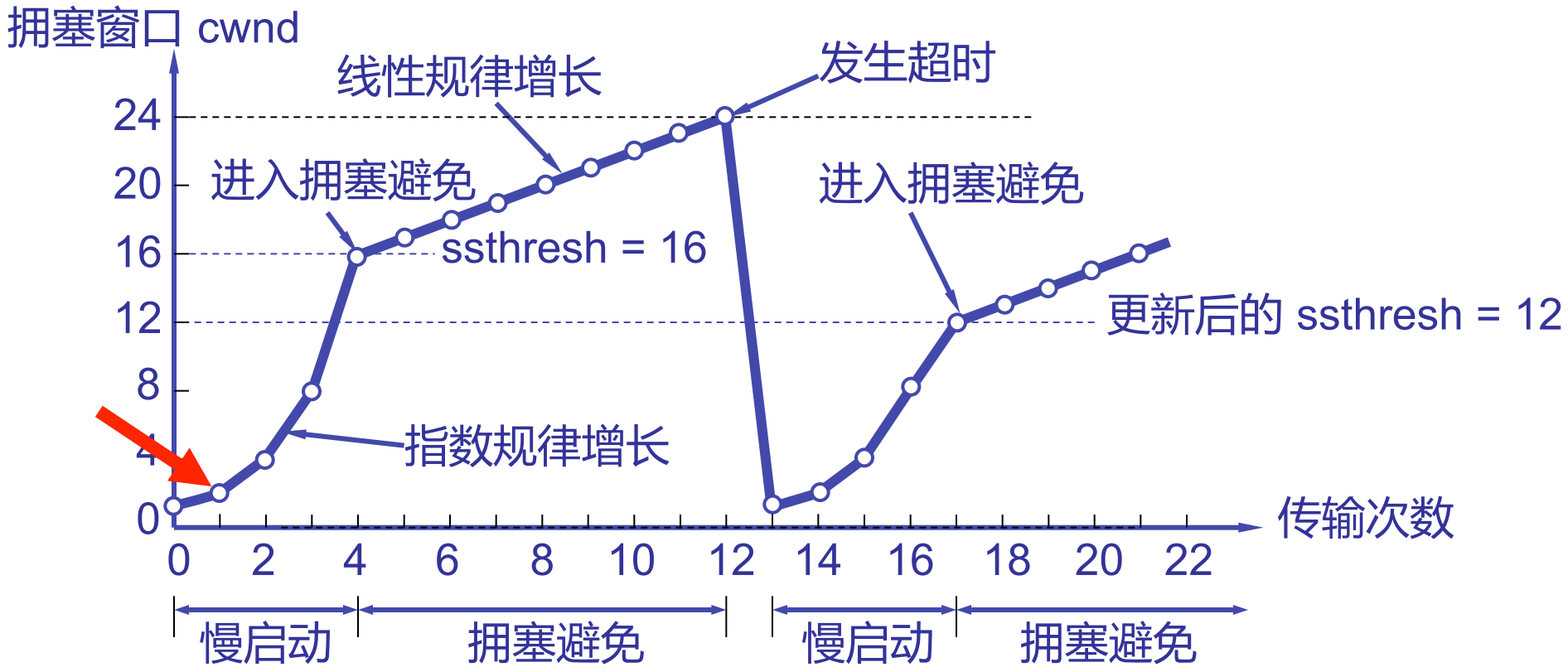
发送端的发送窗口不能超过拥塞窗口 cwnd 和接收端窗口 rwnd 中的最小值。我们假定接收端窗口足够大，因此现在发送窗口的数值等于拥塞窗口的数值。

慢启动和拥塞避免算法的实现举例



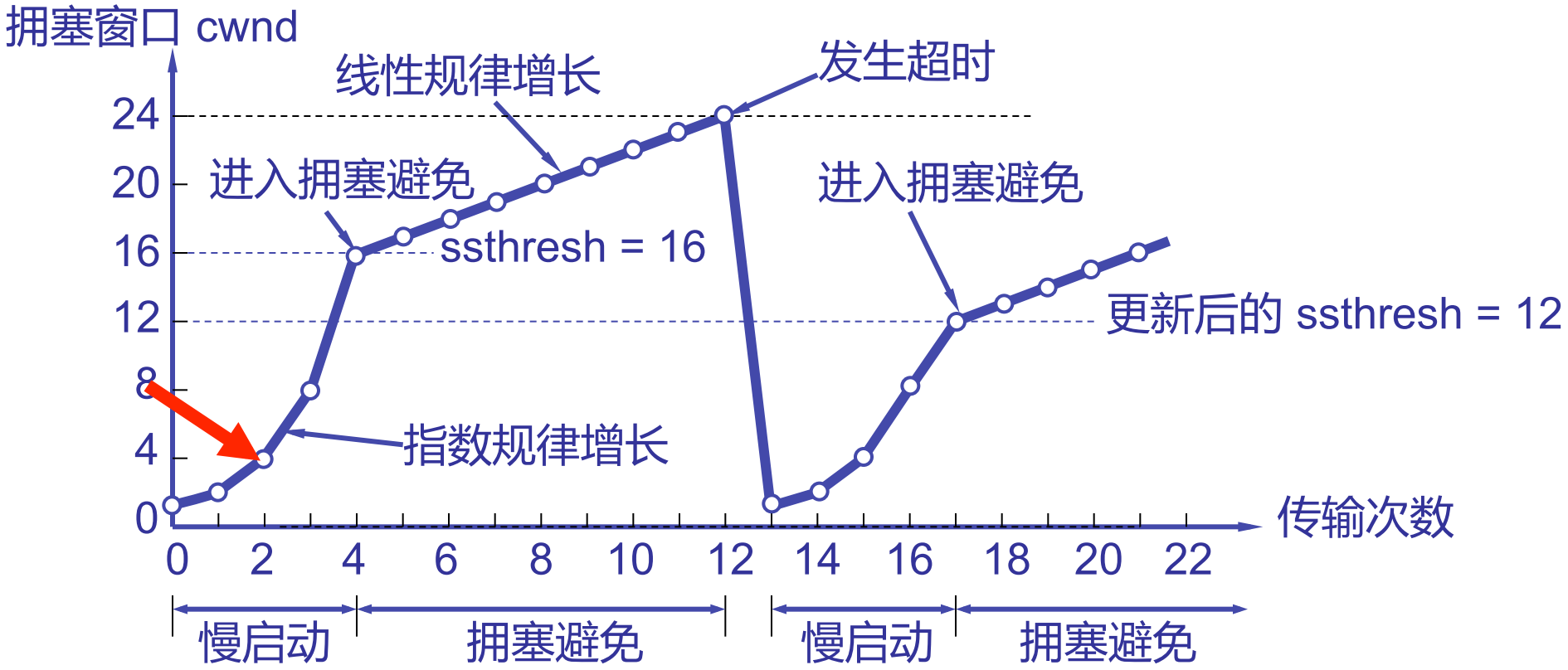
在执行慢启动算法时，拥塞窗口 cwnd 的初始值为 1，发送第一个报文段 M_0。

慢启动和拥塞避免算法的实现举例



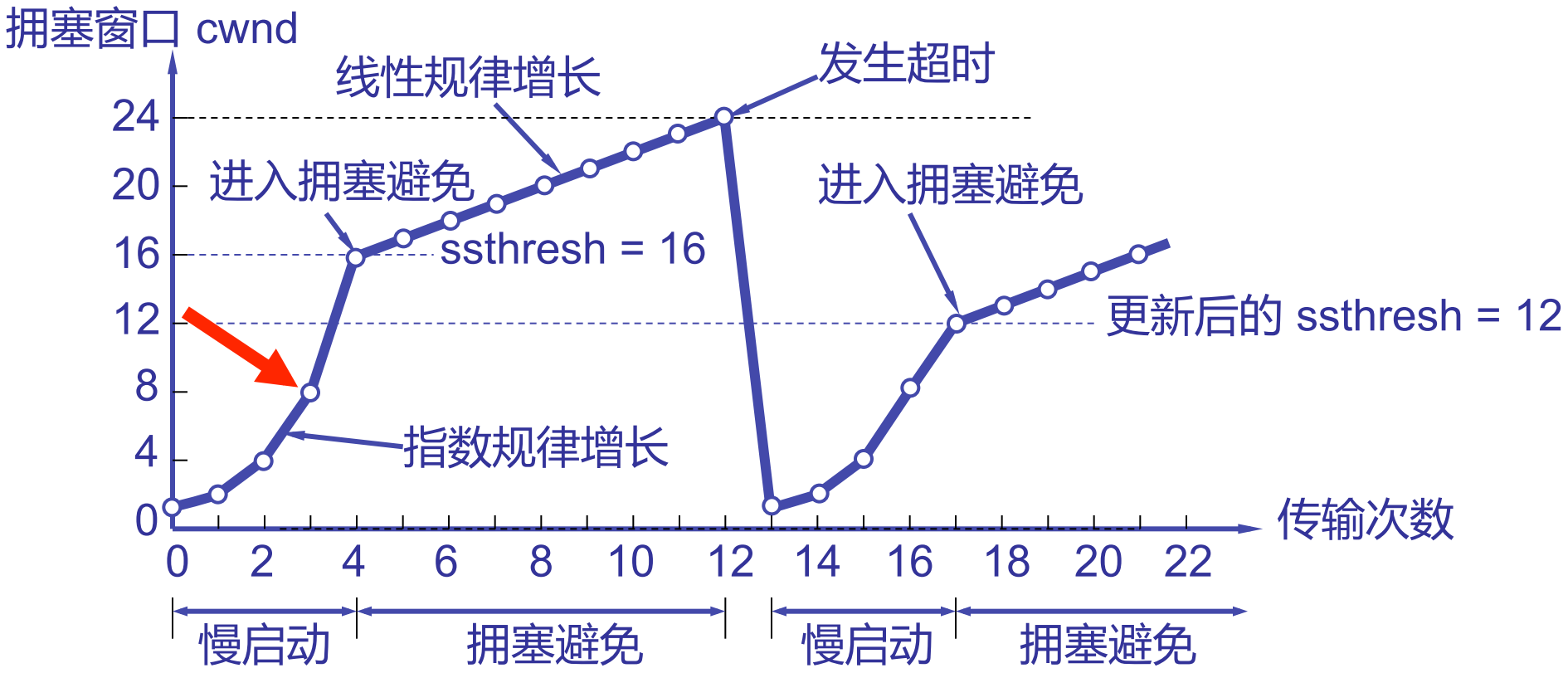
发送端收到 ACK_1（确认 M_0，期望收到 M_1）后，将 cwnd 从 1 增大到 2，于是发送端可以接着发送 M_1 和 M_2 两个报文段。

慢启动和拥塞避免算法的实现举例



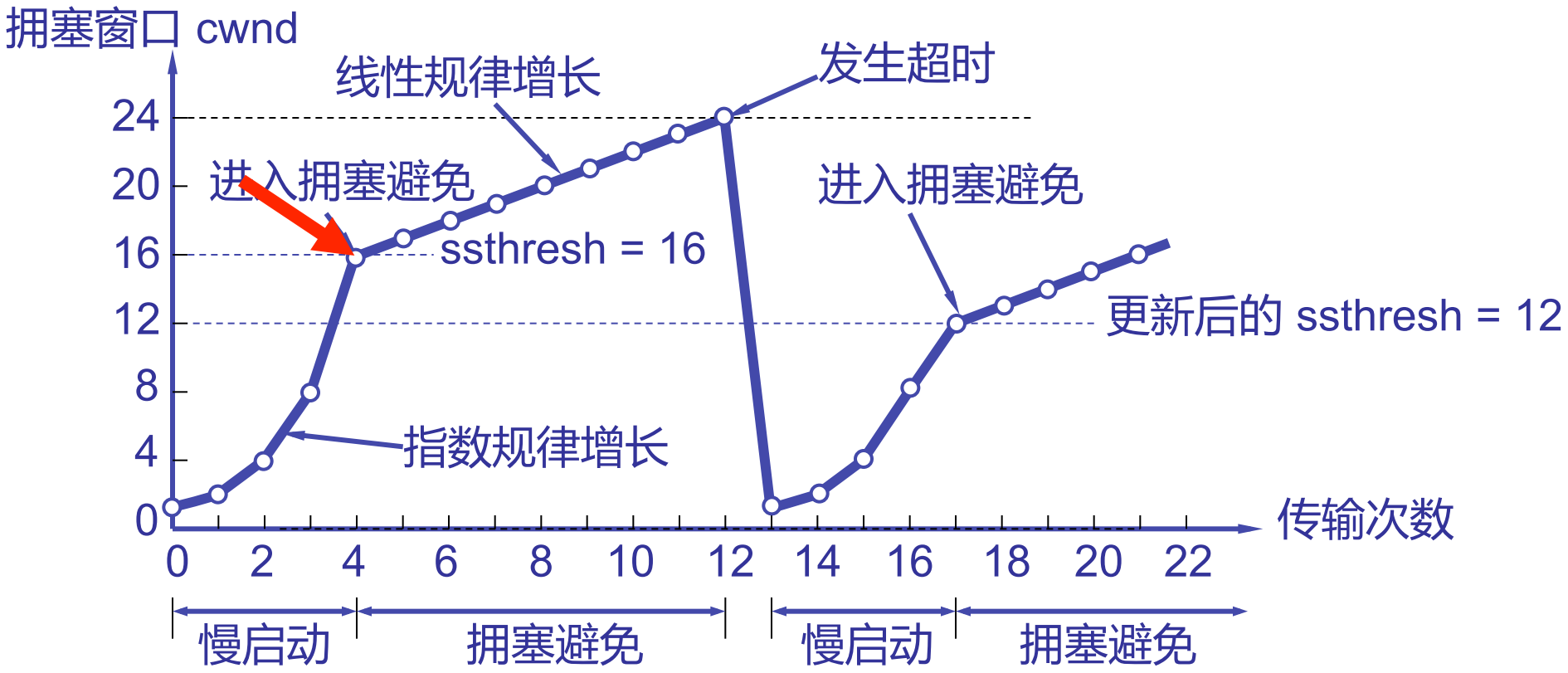
接收端发回 ACK_2 和 ACK_3。发送端每收到一个对新报文段的确认 ACK，就把发送端的拥塞窗口加倍。现在发送端的 cwnd 从 2 增大到 4，并可发送 M_4 ~ M_6共 4 个报文段。

慢启动和拥塞避免算法的实现举例



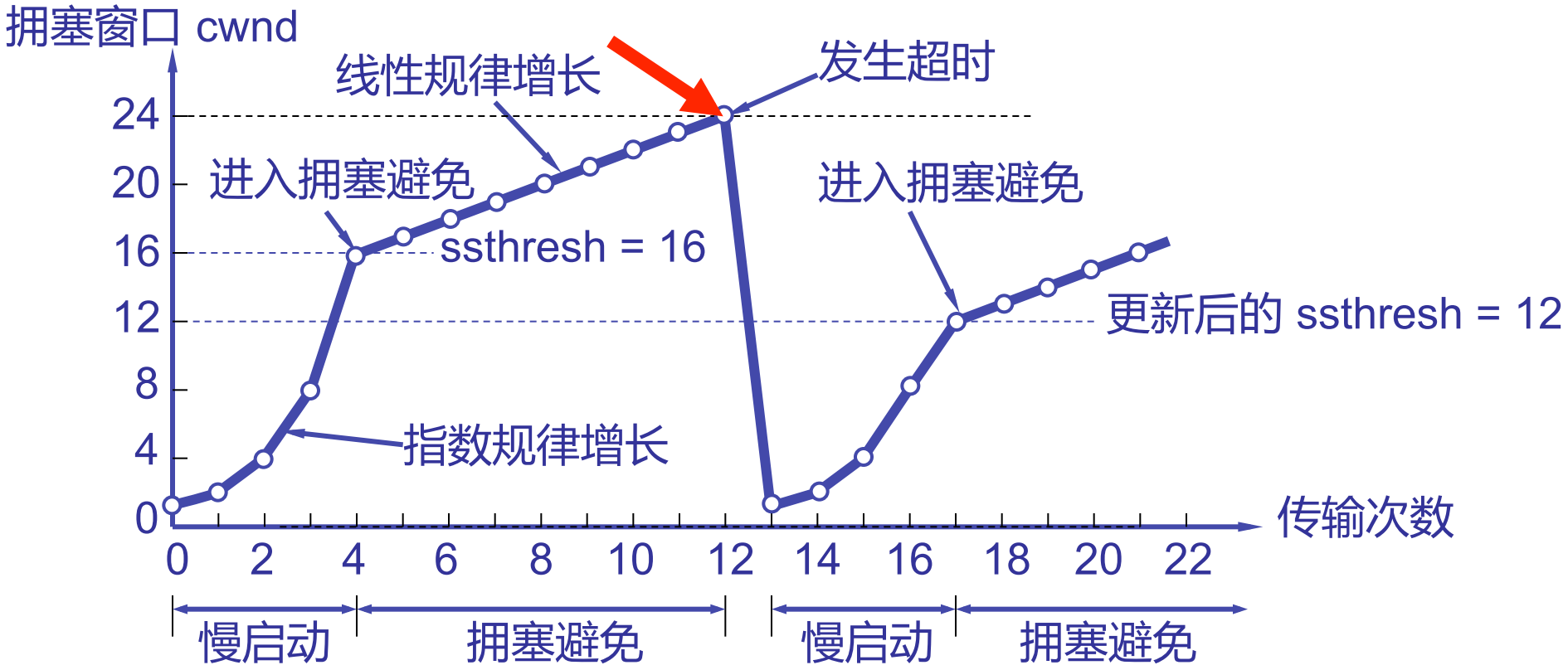
发送端每收到一个对新报文段的确认 ACK，就把发送端的拥塞窗口加倍，因此拥塞窗口 cwnd 随着传输次数按指数规律增长。

慢启动和拥塞避免算法的实现举例



当拥塞窗口 cwnd 增长到慢启动门限值 ssthresh 时（即当 cwnd = 16 时），就改为执行拥塞避免算法，拥塞窗口按线性规律增长，窗口每次增长1。

慢启动和拥塞避免算法的实现举例



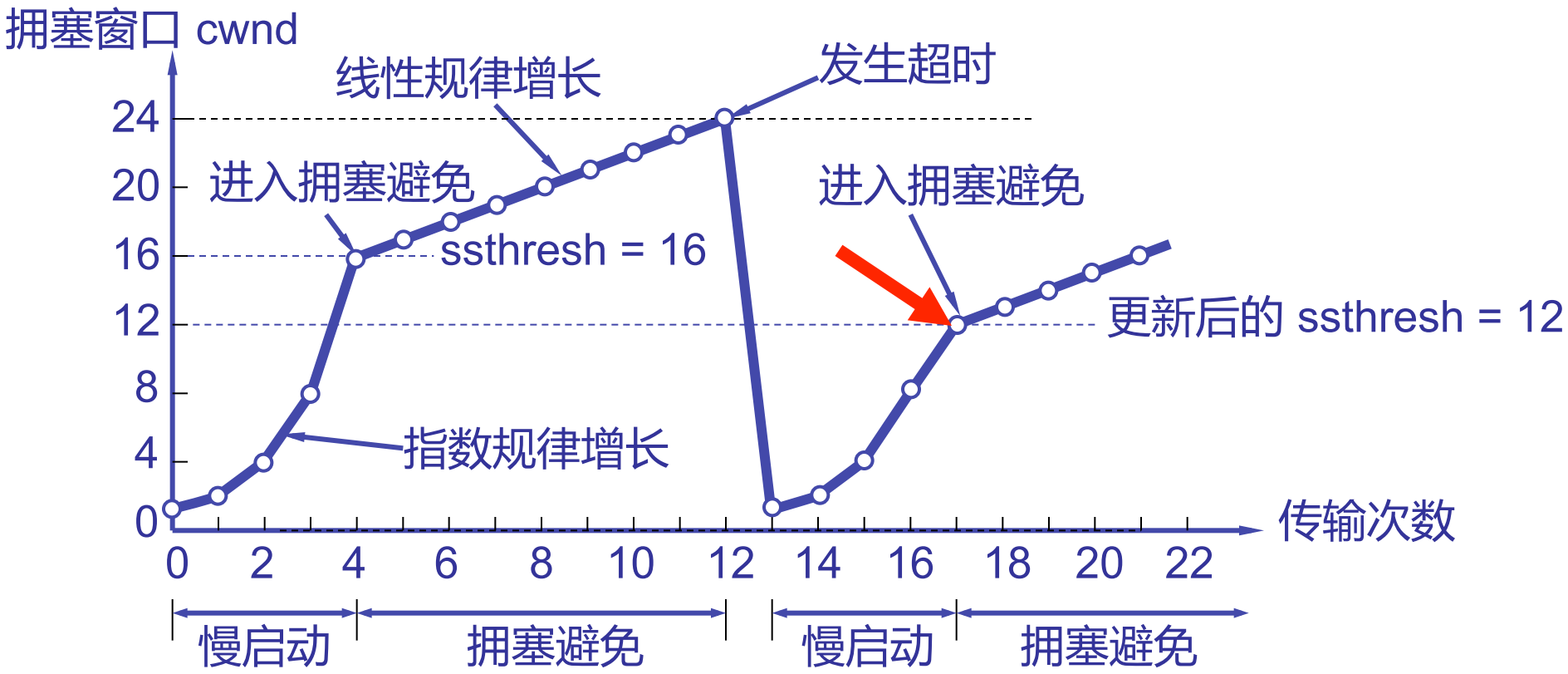
假定拥塞窗口的数值增长到 24 时，网络出现超时（表明网络拥塞了）。

慢启动和拥塞避免算法的实现举例



更新后的 ssthresh 值变为 12（即发送窗口数值 24 的一半），拥塞窗口再重新设置为 1，并执行慢启动算法。

慢启动和拥塞避免算法的实现举例



当 cwnd = 12 时改为执行拥塞避免算法，拥塞窗口按按线性规律增长，每经过一个往返时延就增加一个 MSS 的大小。

乘法递减(multiplicative decrease)

- “乘法递减”是指不论在慢启动阶段还是拥塞避免阶段，只要出现一次超时（即出现一次网络拥塞），就把慢启动门限值 ssthresh 设置为当前的拥塞窗口值乘以 0.5。
- 当网络频繁出现拥塞时，ssthresh值就下降得很快，以大大减少注入到网络中的分组数。

加法递增(additive increase)

- “加法递增”是指执行拥塞避免算法后，当收到对所有报文段的确认就将拥塞窗口 cwnd 增加一个MSS大小，使拥塞窗口缓慢增大，以防止网络过早出现拥塞。

习题

- 第6章习题15，17，18，23，28，31，32，33